IUT de Rodez - Département Informatique
R3.09 - Cryptographie et sécurité

⋆———————————⋆

Table des matières

1 Introduction **5**
1.1 Définitions . . . 5
1.2 Techniques de chiffrement . . . 5
1.3 Les acteurs de la cryptographie . . . 6
1.4 Aperçu historique . . . 6
1.4.1 La scytale . . . 6
1.4.2 Le chiffre de César . . . 7
1.4.3 Le chiffre des templiers . . . 7
1.4.4 Le chiffre de Vigenère . . . 8
1.4.5 Les machines à chiffrer : ENIGMA . . . 9
1.4.6 Le masque de Vernam . . . 9

2 Outils mathématiques indispensables **11**
2.1 Les ensembles $\mathbb{Z}/n\mathbb{Z}$ . . . 11
2.1.1 Rappels . . . 11
2.1.2 Applications . . . 12
2.1.3 Fonction d'Euler et théorème d'Euler . . . 13
2.2 Générateurs de nombres pseudo aléatoires . . . 14
2.2.1 Principes . . . 14
2.2.2 La période . . . 15
2.2.3 Exemple . . . 15
2.3 Les fonctions à sens unique . . . 15
2.3.1 Factorisation d'un entier . . . 16
2.3.2 Logarithme discret . . . 16
2.4 Fonction de hachage . . . 17
2.4.1 Principe d'une fonction de hachage . . . 17
2.4.2 Propriétés requises pour une fonction de hachage . . . 17
2.4.3 Exemple d'utilisation : protection par mot de passe . . . 17

3 Initiation à la cryptographie symétrique **18**
3.1 Principes généraux . . . 18
3.2 Chiffrement par substitution mono-alphabétique . . . 18
3.2.1 Principe . . . 18
3.2.2 Chiffrement affine . . . 19
3.2.3 Permutation aléatoire . . . 19
3.2.4 Cryptanalyse par analyse des fréquences . . . 20
3.3 Chiffrement par substitution poly-alphabétique . . . 20
3.3.1 Chiffrement de Vigenère . . . 20
3.3.2 Chiffrement de Hill . . . 23
3.3.3 Chiffrement de Vernam : technique du masque jetable . . . 24
3.3.4 Chiffrement à flot de Vernam . . . 25
3.4 Le DES : Data Encryption Standard . . . 25

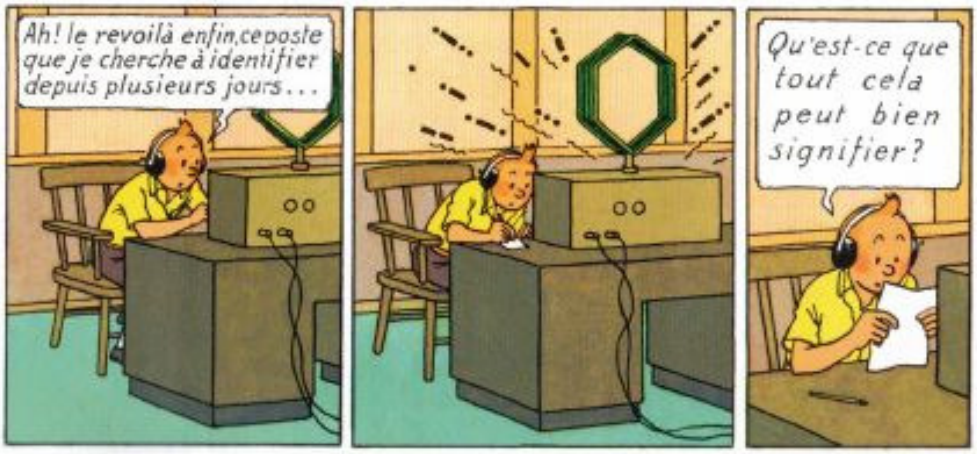
4 Initiation à la cryptographie asymétrique **26**
4.1 Principes généraux . . . 26
4.2 Le chiffrement RSA . . . 27
4.2.1 Mise en place d'un système RSA . . . 27
4.2.2 Comment chiffrer un message . . . 27
4.2.3 Comment déchiffrer un message . . . 27
4.2.4 Un exemple . . . 28
4.2.5 Un autre exemple . . . 28
4.2.6 Preuve du système RSA . . . 30
4.2.7 Sécurité du système RSA . . . 30
4.3 Signature numérique d'un document . . . 30
4.4 Echange de Diffie-Hellman . . . 31
4.4.1 Principe de l'échange . . . 31
4.4.2 Comment trouver cette donnée secrète ? . . . 32
4.4.3 Un exemple . . . 32
4.5 Procédé d'El Gamal . . . 32
4.5.1 Principe . . . 32
4.5.2 Un exemple . . . 33

5 Codes détecteurs et correcteurs **34**
5.1 Schéma général d'une transmission de données . . . 35
5.2 L'idée de base des codes détecteurs et correcteurs . . . 35
5.3 Notion de distance de Hamming . . . 36
5.3.1 Mot binaire . . . 36
5.3.2 Addition de deux mots binaires . . . 36
5.3.3 Distance de Hamming . . . 37
5.4 Codage par bloc . . . 37
5.5 Le codage, la transmission, le décodage : principes généraux . . . 39
5.5.1 Le codage . . . 39
5.5.2 La transmission . . . 39
5.5.3 Le décodage . . . 39
5.5.4 Principe de la correction . . . 40
5.6 Distance minimale d'un code . . . 40
5.6.1 Définition . . . 40
5.6.2 Que se passe-t-il s'il y a $N = 1$ erreur lors de la transmission . . . 41
5.6.3 Cas général . . . 42
5.6.4 Qualité d'un bon code . . . 42
5.6.5 Borne de Hamming . . . 42

6 Codages linéaires **44**
6.1 Définition et représentation matricielle . . . 44
6.1.1 Définition et propriétés . . . 44
6.1.2 Intérêt d'un codage linéaire . . . 44
6.1.3 Représentation matricielle d'un codage linéaire . . . 45

6.2 Matrice de contrôle et syndrome d'un message 46
6.2.1 Matrice de contrôle 46
6.2.2 Syndrome d'un message 46
6.3 Détection et correction d'un message par syndrome 47
6.3.1 Détection d'une erreur 47
6.3.2 Correction d'un message par syndrome 48
6.4 Code de Hamming 49
6.5 Synthèse sur les erreurs 49

7 Eléments de bibliographie **51**



1 Introduction

1.1 Définitions

La cryptologie C'est la science qui étudie les outils servant à dissimuler des informations et à transmettre des messages en toute confidentialité. Ce terme vient du grec : *kruptos* = secret et *logos* = discours.

La cryptologie englobe **la cryptographie** et **la cryptanalyse**.

La cryptographie L'objectif est de concevoir des systèmes visant à assurer la sécurité des communications sur un canal public, susceptible d'espionnage. La cryptographie vise donc à transformer un message clair en un message inintelligible pour une personne non habilitée à recevoir ce message.

La cryptanalyse L'objectif est ici de trouver des failles dans un cryptosystème (c'est à dire un système cryptographique).

1.2 Techniques de chiffrement

Le chiffrement regroupe les techniques utilisées pour brouiller la signification d'un message qui est matériellement visible. Le contenu ne doit être retrouvé que par les personnes destinataires du message.

Le chiffrement fait appel à deux procédés impliquant la transformation des lettres d'un message :

- La substitution qui consiste à remplacer, *sans en modifier l'ordre*, les symboles d'un texte clair par d'autres symboles.
  Exemple : Le texte *JE SUIS EN BUT INFORMATIQUE* devient, une fois chiffré,
  MHVXLVHQEXWLQIRUPDWLTXH (on a retiré les espaces)

  C'est un chiffrement de César (simple décalage de 3 lettres).

- La transposition qui consiste *à modifier l'ordre* des symboles d'un texte clair.
  Exemple : Le texte *JE SUIS EN BUT INFORMATIQUE* devient, une fois chiffré,
  EUQITAMROFNI TUB NE SIUS EJ

1.3 Les acteurs de la cryptographie

Nous n'identifierons ici que trois des principaux acteurs de la cryptologie : Alice, Bob et Carole.

Alice veut transmettre un message à Bob, et seulement à Bob, via un canal qui n'est pas sécurisé.
Elle doit donc au préalable transformer son message de sorte que :

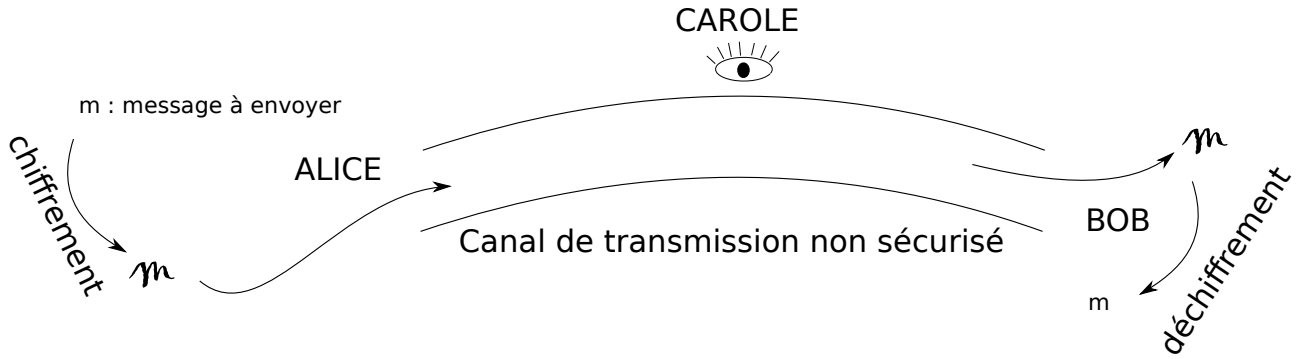
⟶ Un observateur extérieur ne puisse pas le comprendre.

⟶ Bob puisse déchiffrer le message et le comprendre.

Alice et Bob peuvent être deux personnes physiques, ou des machines, ou un mélange des deux.

Carole, quant à elle, est l'espion qui écoute ce que se disent Alice et Bob. Ce peut aussi être une personne physique ou une machine.

Quand Carole se contente d'écouter, on dit que c'est un attaquant passif.
Mais Carole peut très bien être l'auteur d'un "faux" message. Dans ce cas, c'est un attaquant actif.



1.4 Aperçu historique

La cryptographie a été utilisée de tout temps, et notamment pendant les conflits militaires, pour transmettre un message sans que celui-ci ne puisse être compris par l'adversaire.
La plupart des méthodes présentées dans ce paragraphe ne sont plus guère utilisées aujourd'hui : la puissance de calcul, toujours accrue, des ordinateurs rend ces méthodes peu fiables.

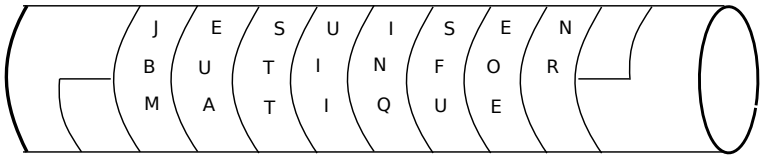
1.4.1 La scytale

La scytale est un exemple de chiffrement par transposition. Elle a été utilisée à partir de 600 ans avant JC en Grèce.
Le principe consiste à enrouler une bande de papyrus sur un cylindre appelé scytale. On écrit ensuite le message à transmettre (une lettre par largeur de papyrus).
La bande de papyrus, une fois déroulée, donne un texte chiffré.
Pour déchiffrer, il faut utiliser un cylindre de même diamètre que celui utilisé pour le chiffrement.

Voici un exemple de chiffrement par scytale.

Le texte *JE SUIS EN BUT INFORMATIQUE* devient, une fois les espaces retirés, *JESUISENBUTINFORMATIQUE*.

La figure ci-dessous explique le principe du chiffrement par scytale. Le papyrus a été déroulé autour du cylindre. Le message à chiffrer a été écrit, une lettre par largeur de papyrus.



Finalement, le chiffrement par scytale donnera *JBMEUASTTUIIINQSFUEOENR*

Globalement, les lettres du messages en clair sont conservées ; mais leur ordre a changé.

Le chiffrement par scytale dépend :

- du diamètre de la scytale,
- de la longueur du papyrus.

1.4.2 Le chiffre de César

Jules César a laissé son nom à un chiffrement par substitution, basé sur un décalage de trois rangs dans l'ordre alphabétique (les trois dernières lettres de l'alphabet étant remplacé par les trois premières).

Procédé de chiffrement

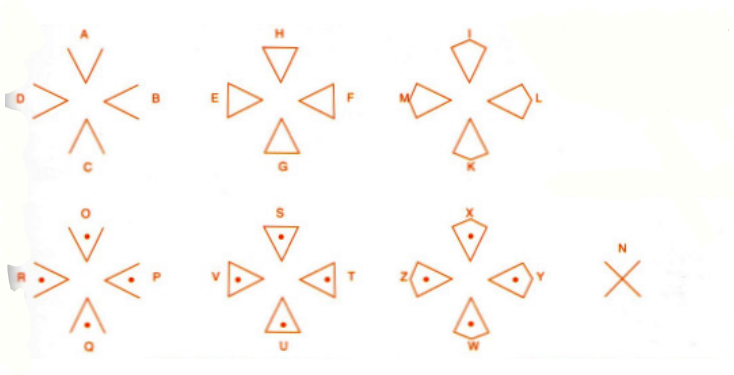
a est chiffrée en d
b est chiffrée en e
c est chiffrée en f
...
w est chiffrée en z
x est chiffrée en a
y est chiffrée en b
z est chiffrée en c

Ce procédé, ou des procédés similaires avec un décalage différent de trois, ont en réalité été utilisés plusieurs siècles avant Jules César.

1.4.3 Le chiffre des templiers

Utilisé par l'ordre des templiers pour transmettre leurs lettres de crédit, le chiffre des templiers est un procédé par substitution : chaque lettre est ici remplacée par un symbole.

Les correspondances entre les lettres et les symboles sont données ci-dessous.



Lettres	Symboles de chiffrement
A	⋁
B	<
C	⋀
D	>
E	▷
F	◁
G	△
H	▽
...	...

1.4.4 Le chiffre de Vigenère

Mis au point par Blaise de Vigenère (1523-1596) au 16ème siècle, le chiffre de Vigenère est un chiffrement par substitution poly-alphabétique. Un tel procédé consiste à remplacer un symbole du texte texte clair par un autre *qui dépend de l'état du cryptosystème*. Cette modification de la substitution dépend donc de la position du symbole dans le message.

Le chiffre de Vigenère repose sur le choix d'une clé (en général un mot). Pour chiffrer un texte clair, à chaque caractère nous associons une lettre de la clé et nous effectuons un décalage dépendant de la lettre de la clé.

A titre d'exemple, chiffrons le message *JE SUIS EN BUT INFORMATIQUE*, avec la clé *RODEZ*.
Nous supprimons les espaces, le message devient *JESUISENBUTINFORMATIQUE*
Nous mettons et répétons la clé autant de fois que nécessaire, en vis à vis du message.
Les lettres de la clé donnent alors le décalage à appliquer pour chiffrer.

Ainsi, le R de *RODEZ* donne un décalage de 17. Il s'ensuit que le J est décalé de 17 et est donc chiffré en A.
Le O de *RODEZ* donne un décalage de 14. Il s'ensuit que le E est décalé de 14 et est donc chiffré en S.

Le tableau suivant donne le chiffrement complet.

message	J	E	S	U	I	S	E	N	B	U	T	I	N	F	O	R	M	A	T	I	Q	U	E
clé clé clé ...	R	O	D	E	Z	R	O	D	E	Z	R	O	D	E	Z	R	O	D	E	Z	R	O	D
chiffrement	A	S	V	Y	H	J	S	Q	F	T	K	W	Q	J	N	I	A	D	X	H	H	I	H

Finalement, le chiffrement obtenu est *ASVYHJSQFTKWQJNIADXHHIH*

Mis au point en 1586, le chiffre de Vigenère résista à toutes les attaques jusqu'au milieu du 19ème siècle : les travaux, indépendants, de l'anglais Charles Babbage (1792-1871) et du prussien Friedrich Kasiski (1805-1881) ont alors mis un terme à la réputation d'invulnérabilité du chiffre de Vigenère.

1.4.5 Les machines à chiffrer : ENIGMA

Le XXème siècle voit l'avènement de dispositifs électromécaniques destinés à automatiser le chiffrement. L'un de ces procédés, baptisé Enigma, d'une redoutable complexité, mobilisa contre lui pendant près de vingt ans les meilleurs spécialistes en cryptanalyse.
La machine Enigma tient une place à part dans l'histoire de la cryptologie : complexité du chiffrement, cryptanalyse avec de sérieux moyens mathématiques et informatiques ... et avec aussi de la chance.

La machine a été mise au point à partir de 1918 ; l'armée allemande s'en dotait rapidement, soucieuse de bénéficier d'un système sûr de cryptographie. Elle a été utilisée de façon intensive par l'Allemagne nazie, indispensable notamment pour la transmission des communications lors des phases de "guerre éclair" ("Blitzkrieg") durant lesquelles blindés, infanterie et aviation devaient intervenir de façon coordonnée. L'état-major allemand avait en Enigma une confiance absolue.

La machine Enigma.

Les trois rotors,
permettant de complexes substitutions,
sont visibles.

Les lampes indiquent
la lettre qui se substitue
à celle actionnée
sur le clavier.



Les polonais puis les britanniques (Alan Turing), aidés par les progrès technologiques alors en plein essor, eurent raison de la complexité d'Enigma : les messages transmis pouvaient en partie être décryptés et cela a contribué à la victoire des alliés sur l'Allemagne nazie.

1.4.6 Le masque de Vernam

Cette technique, mise au point par Gilbert Vernam au début du XXème siècle, en 1918, s'inspire du chiffre de Vigenère mais présente quelques caractéristiques essentielles qui en font un système parfaitement sécurisé.

Caractéristique du chiffre de Vernam :

- la clé est constituée d'une liste *aléatoire* de nombres compris entre 0 et 25
  (ou bien d'une suite de lettres, chaque lettre correspondant à un nombre entre 0 et 25),
- la clé doit avoir *la même longueur* que le message en clair,
- la clé ne doit être utilisée *qu'une seule fois*.

On parle alors de masque (la clé) jetable (à détruire une fois utilisé). Sous ces conditions, cette technique est parfaitement sécurisée.

A titre d'exemple, chiffrons le message *JE SUIS EN BUT INFORMATIQUE*.
Nous supprimons les espaces, le message devient *JESUISENBUTINFORMATIQUE*
Le masque choisi aléatoirement est *ASDEZTAKKIANMPAJHYZSYZE*; il a la même longueur que le message en clair.
Nous mettons le masque en vis à vis du message.
Les lettres du masque donnent alors le décalage à appliquer pour chiffrer.

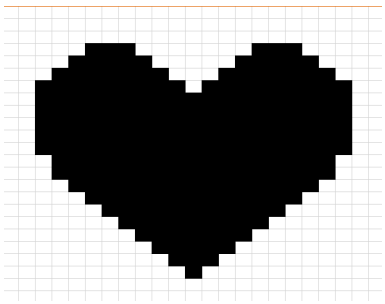
Le tableau suivant donne le chiffrement complet.

message	J	E	S	U	I	S	E	N	B	U	T	I	N	F	O	R	M	A	T	I	Q	U	E
clé clé clé ...	A	S	D	E	Z	T	A	K	K	I	A	N	M	P	A	J	H	Y	Z	S	Y	Z	E
chiffrement	J	W	V	Y	H	L	E	X	L	C	T	V	Z	U	O	A	T	Y	S	A	O	T	I

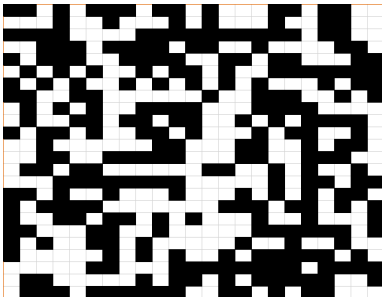
Finalement, le chiffrement obtenu est *JWVYHLEXLCTVZUOATYSAOTI*

Le masque (jetable) de Vernam peut aussi être utilisé pour la transmission sécurisée d'une image : on parle alors de **cryptographie visuelle**. Un exemple est proposé ci-dessous :

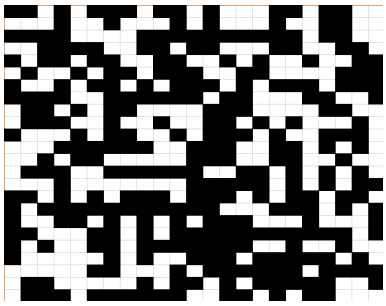
L'image, en claire, à transmettre



Le masque (choisi aléatoirement)



L'image chiffrée



2 Outils mathématiques indispensables

2.1 Les ensembles $\mathbb{Z}/n\mathbb{Z}$

2.1.1 Rappels

Définition

Sur $\mathbb{Z}$ on définit la relation de congruence modulo un entier $n \geq 2$.
Cette relation de congruence est une relation d'équivalence.

$\mathbb{Z}/n\mathbb{Z}$ est l'ensemble quotient, c'est à dire l'ensemble des classes d'équivalence.

Exemple : $\mathbb{Z}/5\mathbb{Z} = \{\overline{0},\ \overline{1},\ \overline{2},\ \overline{3},\ \overline{4}\}$

$\overline{0}$ est la classe d'équivalence de 0 ; elle est constituée de tous les entiers congrus à 0 modulo 5
$\overline{0} = \{..., -10, -5, 0, 5, 10, 15, ...\} = \{5k \ tel \ que \ k \in \mathbb{Z}\}$

$\overline{1}$ est la classe d'équivalence de 1 ; elle est constituée de tous les entiers congrus à 1 modulo 5
$\overline{1} = \{..., -9, -4, 1, 6, 11, 16, ...\} = \{5k + 1 \ tel \ que \ k \in \mathbb{Z}\}$

etc ...

$\overline{4}$ est la classe d'équivalence de 4 ; elle est constituée de tous les entiers congrus à 4 modulo 5
$\overline{4} = \{..., -6, -1, 4, 9, 14, 19, ...\} = \{5k + 4 \ tel \ que \ k \in \mathbb{Z}\}$

Dans la suite, on écrira simplement x au lieu de $\overline{x}$ pour désigner la classe de x dans $\mathbb{Z}/n\mathbb{Z}$.

Opérations dans $Z/n\mathbb{Z}$

$Z/n\mathbb{Z}$ est naturellement muni de deux opérations : une addition et une multiplication.

A titre d'exemple, voici ci-dessous les tables d'addition et de multiplication dans $Z/5\mathbb{Z}$.

+	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

×	0	1	2	3	4
0	0	0	0	0	0
1	0	1	2	3	4
2	0	2	4	1	3
3	0	3	1	4	2
4	0	4	3	2	1

Eléments inversibles dans $\mathbb{Z}/n\mathbb{Z}$

Nous rappelons ici un résultat essentiel concernant l'inversibilité d'une classe de $\mathbb{Z}/n\mathbb{Z}$ vis à vis de la multiplication.

$$\overline{x} \in \mathbb{Z}/n\mathbb{Z} \ inversible \ \Leftrightarrow \exists\ \overline{y} \in \mathbb{Z}/n\mathbb{Z} \ tq \ \overline{xy} = \overline{1}$$

$$\Leftrightarrow xy \equiv 1[n] \Leftrightarrow xy + kn = 1 \Leftrightarrow x \ et \ n \ premiers \ entre \ eux$$

Ainsi, une classe $\overline{x} \in \mathbb{Z}/n\mathbb{Z}$ est inversible
$\Leftrightarrow$
x et n sont deux entiers premiers entre eux.

Exemple : dans $\mathbb{Z}/105\mathbb{Z}$, nous allons montrer que $\overline{16}$ est inversible et nous déterminerons sa classe inverse.

- On cherche $pgcd(105, 16)$ $\longrightarrow$ algorithme d'Euclide

$$105 = 16 \times 6 + 9$$

$$16 = 9 \times 1 + 7$$

$$9 = 7 \times 1 + 2$$

$$7 = 2 \times 3 + 1$$

on a $pgcd(105, 16) = 1$ donc 105 et 16 sont premiers entre eux

Dans $\mathbb{Z}/105\mathbb{Z}$, $\overline{16}$ est inversible : modulo 105, 16 est inversible.

- On cherche deux entiers u et v tels que $105u + 16v = 1$ $\longrightarrow$ algorithme d'Euclide étendu

$$1 = 7 - 2 \times 3$$

$$1 = 7 - (9 - 7 \times 1) \times 3 = 7 \times 4 - 9 \times 3$$

$$1 = (16 - 9 \times 1) \times 4 - 9 \times 3 = 16 \times 4 - 9 \times 7$$

$$1 = 16 \times 4 - (105 - 16 \times 6) \times 7 = 16 \times 46 - 105 \times 7$$

$$d'où\ 16 \times 46 \equiv 1[105]$$

$\overline{46}$ est la classe inverse de $\overline{16}$

Le groupe $((\mathbb{Z}/p\mathbb{Z})^*, \times)$

Si p est un entier premier, alors toutes les classes de $\mathbb{Z}/p\mathbb{Z}$ (sauf celle de 0) sont inversibles.
Dans ce cadre, $((\mathbb{Z}/p\mathbb{Z})^*, \times)$ est **un groupe abélien**.

2.1.2 Applications

Résolution d'une équation linéaire dans $\mathbb{Z}/n\mathbb{Z}$

Exemple 1 : résolvons, dans $\mathbb{Z}/5\mathbb{Z}$, l'équation $2x + 4 \equiv 3[5]$
Dans $\mathbb{Z}/5\mathbb{Z}$ on a $2x + 4 = 3 \Leftrightarrow 2x = 4 \Leftrightarrow x = 3 \times 4 = 12 = 2$
(Dans $\mathbb{Z}/5\mathbb{Z}$, 2 est inversible et son inverse est 3.)

Exemple 2 : résolvons, dans $\mathbb{Z}/16\mathbb{Z}$, l'équation $11x + 3 \equiv 10[16]$
Dans $\mathbb{Z}/16\mathbb{Z}$ on a $11x + 3 = 10 \Leftrightarrow 11x = 7 \Leftrightarrow x = 3 \times 7 = 21 = 5$
(Dans $\mathbb{Z}/16\mathbb{Z}$ la classe inverse de 11 est 3.)

Inversion d'une matrice modulo n

Exemple 1 : modulo 5, inverser la matrice $A = \begin{pmatrix} 3 & 1 \\ 3 & 2 \end{pmatrix}$

On a $\det A = 3 \times 2 - 3 \times 1 = 3$

$\det A = 3$ est inversible modulo 5 donc A est inversible. L'inverse de $\det A$ est 2.

On obtient $A^{-1} = 2 \begin{pmatrix} 2 & -1 \\ -3 & 3 \end{pmatrix} = 2 \begin{pmatrix} 2 & 4 \\ 2 & 3 \end{pmatrix} = \begin{pmatrix} 4 & 8 \\ 4 & 6 \end{pmatrix} = \begin{pmatrix} 4 & 3 \\ 4 & 1 \end{pmatrix}$

On peut vérifier que $A\ A^{-1} = \begin{pmatrix} 3 & 1 \\ 3 & 2 \end{pmatrix} \begin{pmatrix} 4 & 3 \\ 4 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ (en faisant des calculs modulo 5)

Exemple 2 : modulo 5, inverser la matrice $A = \begin{pmatrix} 4 & 1 \\ 3 & 2 \end{pmatrix}$

On a $\det A = 4 \times 2 - 3 \times 1 = 5 = 0$

$\det A = 0$ n'est pas inversible modulo 5 donc A est singulière (dans $\mathbb{Z}/5\mathbb{Z}$).

Exemple 3 : modulo 5, inverser la matrice $A = \begin{pmatrix} 4 & 1 & 0 \\ 1 & 0 & 3 \\ 2 & 1 & 2 \end{pmatrix}$

Le calcul du détermination de A (par exemple avec une règle de Sarrus) donne $\det A = 2$

Or 2 est une classe inversible donc A est une matrice inversible.

La recherche de l'inverse de A (par exemple avec une méthode de Gauss) donne $A^{-1} = \begin{pmatrix} 1 & 4 & 4 \\ 2 & 4 & 4 \\ 3 & 4 & 2 \end{pmatrix}$

Calcul d'une exponentiation modulaire

Dans $\mathbb{Z}/n\mathbb{Z}$, on cherche à calculer une expression du type x^a où x est une classe de $\mathbb{Z}/n\mathbb{Z}$ et a est un entier.

Exemple : dans $\mathbb{Z}/17\mathbb{Z}$ on cherche à calculer 5^{13}

On a $5 = 5$ $\quad 5^2 = 25 = 8$ $\quad 5^4 = 8^2 = 64 = 13$ $\quad 5^8 = 13^2 = 169 = -1 = 16$

On vient de calculer 5^k où k est une puissance de 2

On en déduit que $5^{13} = 5 \times 5^4 \times 5^8 = 5 \times 13 \times 16 = 65 \times 16 = 14 \times 16 = 224 = 3$

On s'est appuyé sur la décomposition de 13 en binaire : 13=1+4+8

2.1.3 Fonction d'Euler et théorème d'Euler

La fonction d'Euler

La fonction d'Euler $\varphi(n)$ compte le nombre de classes inversibles dans $\mathbb{Z}/n\mathbb{Z}$.

On a, par exemple :

- $\varphi(4) = 2$ classes inversibles : 1, 3
- $\varphi(5) = 4$ classes inversibles : 1, 2, 3, 4
- $\varphi(10) = 4$ classes inversibles : 1, 3, 7, 9

En fait, $\varphi(n)$ est égal au nombre d'entiers compris entre 1 et $n-1$, et qui sont premiers avec n.

Propriétés :
- Si p est un nombre premier, alors $\varphi(p) = p - 1$; toutes les classes (sauf celle de 0) sont inversibles.
- Si n et m sont deux entiers premiers entre eux, alors $\varphi(nm) = \varphi(n) \times \varphi(m)$

Exemples : $\varphi(6) = \varphi(2) \times \varphi(3) = 1 \times 2 = 2$ $\varphi(26) = \varphi(13) \times \varphi(2) = 12 \times 1 = 12$

Le théorème d'Euler

Théorème d'Euler :

Soit $n \in \mathbb{N}^*$
Soit a un entier **premier avec** n

Alors $a^{\varphi(n)} \equiv 1[n]$

Exemple : $\varphi(17) = 16$; 17 et 5 sont premiers entre eux
Donc dans $\mathbb{Z}/17\mathbb{Z}$ on a $5^{16} = 1$

Cas particulier : le petit théorème de Fermat

Le petit théorème de Fermat est un cas particulier du théorème d'Euler : le n (le modulo) est égal à un entier p premier. Dans ce cas, $\varphi(n) = \varphi(p) = p - 1$

Petit théorème de Fermat :

Soit $p \in \mathbb{N}^*$ un entier premier
Soit a un entier **premier avec** p

Alors $a^{p-1} \equiv 1[p]$

Exemple : soient $p = 11$ (premier) et $a = 2$ a et p sont premiers entre eux

On a $2^{10} \equiv 1[11]$

2.2 Générateurs de nombres pseudo aléatoires

2.2.1 Principes

Etre capable de générer des nombres pseudo-aléatoires est essentiel dans de nombreux domaines, et en particulier en cryptographie.
Produire par le calcul des séquences de nombres qui semblent être le résultat du hasard n'est pas chose simple. Une méthode largement répandue repose sur l'utilisation des suites congruentielles linéaires. C'est ainsi que la plupart des langages de programmation procède lorsque l'on souhaite produire des suites aléatoires.

Expliquons le principe d'un générateur congruentiel linéaire :

- m est un entier assez grand ; les nombres pseudo-aléatoires seront compris entre 0 et $m-1$
- $f(x) = ax + b$ est une fonction affine, qui dépend de deux nombres a et b.
C'est une fonction très simple : important pour assurer la production rapide de nombre pseudo-aléatoires.
- x_0, un entier entre 0 et $m-1$, est choisi (par exemple en fonction de l'état de l'horloge interne).

On construit alors la suite $(x_n)_{n\in\mathbb{N}}$ récurrente d'ordre 1 de la manière suivante :

$$\begin{cases} x_0 \text{ donné} \\ x_{n+1} = f(x) \ mod \ m = a \ x_n + b \ mod \ m \end{cases}$$

La séquence de nombres pseudo-aléatoires proposée est alors : x_0, x_1, x_2, ..., x_n, ...

2.2.2 La période

La construction proposée repose sur :
- des valeurs dans un ensemble de cardinal fini (ici des entiers entre 0 et $m-1$)
- un procédé de récurrence d'ordre 1 permettant de déterminer x_{n+1} à partir de x_n

Le nombre de valeurs possibles étant fini, on est alors assuré de retrouver tôt au tard un nombre qui a déjà été proposé précédemment : un générateur congruentiel linéaire est nécessairement périodique. La période maximale est égale à m.
L'enjeu est de trouver des générateurs de période maximale : il faut alors choisir convenablement les entiers a et b qui définissent la fonction f

2.2.3 Exemple

La fonction f est égale à $f(x) = 3x + 1$
La suite $(x_n)_{n\in\mathbb{N}}$ est définie par $\begin{cases} x_0 = 1 \\ x_{n+1} = f(x) \ mod \ 17 = 3 \ x_n + 1 \ mod \ 17 \end{cases}$

Déterminons les termes successifs de la suite $(x_n)_{n\in\mathbb{N}}$ ainsi que la période.

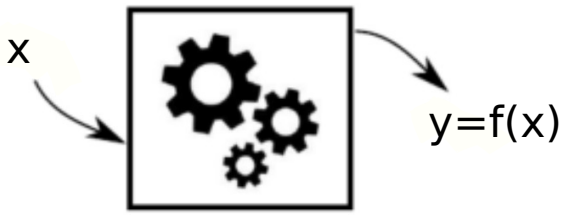
Les termes successifs de la suite sont donnés ci-dessous. La période est égale à 16.

$$1, 4, 13, 6, 2, 7, 5, 16, 15, 12, 3, 10, 14, 9, 11, 0, 1, 4, ...$$

2.3 Les fonctions à sens unique

Une fonction est dite "à sens unique" lorsqu'elle est facilement calculable mais très difficile à inverser. Ces fonctions sont à la base de la cryptographie asymétrique : cryptographie à clé publique.

- $y = f(x)$ est facile à calculer à partir de x
- Une image y étant donnée, il est pratiquement impossible de trouver un antécédent convenable.



2.3.1 Factorisation d'un entier

Un exemple essentiel de fonction à sens unique est donné par la multiplication de deux entiers.

$$x = (a, b) \text{ un couple d'entiers} \qquad y = f(x) = a \times b \quad \text{le produit } ab$$

A partir des entiers a et b, il est facile de déterminer leur produit $a \times b$
Par contre, connaissant un entier y, il est difficile de trouver deux entiers a et b tels que $y = a \times b$

Cette difficulté est d'autant plus grande que les entiers a et b à trouver sont grands. En réalité, cette difficulté est liée au fait qu'on ne connaît (pour l'instant ?) aucun algorithme *efficace* permettant de factoriser un entier.

Ainsi, si je choisis deux entiers premiers p et q *très grands*, je peux facilement les multiplier entre eux et former le produit $n = p \times q$.
Si maintenant je publie l'entier n, *personne ne pourra retrouver les deux entiers p et q* (qui sont les seuls permettant de factoriser n).

Exemple : soit $n = 67030883744037259$ ... quels sont les deux entiers p et q tels que $n = p \times q$?

A titre d'illustration, le tableau suivant donne la date de factorisation d'entiers $n = p \times q$ en fonction du nombre de chiffres dans l'écriture (binaire ou décimale) de n.

Nombre de chiffres dans l'écriture binaire de n	Nombre de chiffres dans l'écriture décimale de n	date de la factorisation de n
332	100	1991
465	140	1999
531	160	2003
770	232	2010
829	250	2020

2.3.2 Logarithme discret

Un autre exemple de fonction à sens unique est donné par le calcul d'une exponentielle modulaire.

$$m \text{ est un entier (le modulo)} \qquad a \text{ est un entier entre } 0 \text{ et } m-1$$

$$x = n \text{ un entier} \qquad y = f(x) = f(n) = a^n \ mod \ m$$

Les entiers m et a sont donnés et connus.
A partir de l'entier n, on calcule a^n modulo m. Ce calcul est facile car il se fait modulo m : après chaque calcul intermédiaire, on ramène le résultat modulo m, donc à une taille raisonnable. Même si n est un entier très grand, un tel calcul ne pose pas de difficulté.
Par contre, connaissant un entier y, il est difficile de trouver un entier n tel que $y = a^n \ mod \ m$
Cette difficulté est d'autant plus grande que l'entier m est grand. Cette difficulté est liée au fait qu'on ne connaît (pour l'instant ?) aucun algorithme *efficace* permettant de trouver n connaissant y.
C'est le problème du logarithme discret.

Exemple : on prend $m = 661643$ et $a = 123101$
Le calcul de $a^{12021} \ mod \ m = 123101^{12021} \ mod \ 661643$ donne 619198. Ce calcul ne pose pas de difficulté.
Par contre, connaissant $y = 619198$, il est difficile de trouver un entier n tel que $y = 123101^n \ mod \ 661643$

2.4 Fonction de hachage

2.4.1 Principe d'une fonction de hachage

Les fonctions de hachage cryptographiques sont des fonction qui associent à un message (une suite de bits de longueur arbitraire) une valeur de longueur fixe (une suite de bits de longueur fixe).

$$\begin{array}{rcccc} h & : & \{0,1\}^* & \rightarrow & \{0,1\}^n \\ & & M & \rightarrow & h(M) \end{array}$$

A partir d'un message M, la fonction de hachage va produire une image $h(M)$ de longueur fixe. On dit que $h(M)$ est le *haché* de M, ou *l'empreinte* de M. En pratique, l'empreinte a une longueur d'au moins 128 bits.

Chaque empreinte numérique se calcule en modifiant, mélangeant, combinant, brassant ... tous les bits du message d'origine. Finalement, l'empreinte d'un message ressemble à une suite qui aurait été obtenue de façon aléatoire. Par ailleurs, la modification d'un seul bit du message doit s'accompagner statistiquement de la modification d'environ la moitié des bits de l'empreinte.

Par construction, une fonction de hachage n'est pas injective : on a réduit l'ensemble d'arrivée par rapport à l'ensemble de départ. Par contre, différents messages donneront presque toujours des empreintes différentes et l'empreinte ne contient pas suffisamment d'information pour permettre la reconstitution du message initial.

Les fonctions de hachage sont utilisées en cryptographie, notamment pour les techniques d'authentification :
- authentification d'un message,
- signature numérique,
- protection de mots de passe.

2.4.2 Propriétés requises pour une fonction de hachage

Pour une utilisation pratique, une fonction de hachage doit posséder les propriétés suivantes :

- il doit être facile de calculer l'empreinte d'un message,
- il doit être très difficile (impossible) de reconstituer un message à partir de son empreinte,
- si on connait l'empreinte d'un message, il doit être très difficile (impossible) de trouver un autre message possédant la même empreinte ; on dit que la fonction h est *résistante aux collisions*.

Les fonctions de hachage utilisées en pratique sont qualifiées de SHA ("Secure Hash Algorithm") : elles produisent des empreintes de 160 bits.

2.4.3 Exemple d'utilisation : protection par mot de passe

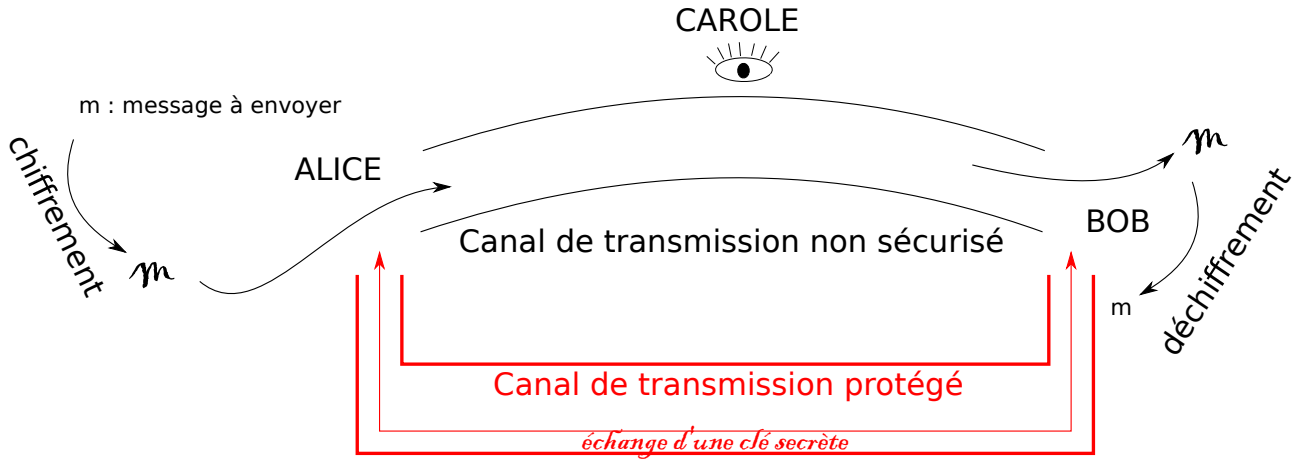
A titre d'illustration, voilà comment fonctionne un protocole d'identification par mot de passe.

Le serveur conserve, non pas la liste des mots de passe en clair, mais seulement la liste de leurs empreintes.
- quand un utilisateur se connecte, il effectue une demande d'accès,
- le serveur demande nom et mot de passe,
- l'utilisateur envoie les renseignements demandés,
- le serveur calcule l'empreinte du mot de passe et la compare à celle stockée dans sa base de données. Si les deux valeurs coïncident, les autorisations sont accordées, sinon, elles sont refusées.

3 Initiation à la cryptographie symétrique

3.1 Principes généraux

La cryptographie symétrique repose sur un échange préalable d'une clé, **sur un canal protégé**, entre l'émetteur et le destinataire. Cette clé permet alors le chiffrement (et le déchiffrement) des messages qui passent, eux, par un canal de transmission non sécurisé.



En fait, la sécurité d'un chiffre ne doit pas reposer sur le secret du procédé, mais seulement sur une clé. Un procédé public, soumis à l'analyse acharnée et vaine de cryptanalystes, sera réputé plus sûr qu'un procédé privé et confidentiel. Rendre le procédé de chiffrement public est finalement un argument de sécurité.

Un procédé de chiffrement symétrique sera considéré comme robuste si, en particulier, il répond au **principe de diffusion**. Le principe de diffusion énonce que les statistiques du message clair doivent se propager sur l'ensemble de tout le message chiffré.
Le respect du principe de diffusion doit conduire, de façon idéale, à un message chiffré statistiquement indiscernable d'une suite aléatoires de symboles.

3.2 Chiffrement par substitution mono-alphabétique

3.2.1 Principe

Un chiffrement par substitution mono-alphabétique consiste à remplacer dans le message en clair chaque symbole, pris individuellement, par un autre symbole.

Le chiffre de César et le chiffre des templiers sont deux exemples de substitution mono-alphabétique.

Dans la suite, nous noterons A l'alphabet utilisé pour écrire les messages. Cet alphabet peut être, tout simplement, constitué des 26 lettres de l'alphabet : $A = \{a, b, c, ..., x, y, z\}$.
Les messages chiffrés utiliseront les mêmes symboles que ceux présents dans l'ensemble A.
Une clé de chiffrement par substitution mono-alphabétique est finalement une bijection de A dans A.

3.2.2 Chiffrement affine

Soit $n = card(A)$: n est le nombre de symboles utilisés pour écrire les messages.
A chaque élément de A, on associera de manière bijective une classe dans $\mathbb{Z}/n\mathbb{Z}$.

La clé de chiffrement consiste en un couple d'entiers (a, b) avec :
- a un élément inversible dans $\mathbb{Z}/n\mathbb{Z}$ Cela assure la bijectivité du chiffre.
- b un élément dans $\mathbb{Z}/n\mathbb{Z}$.

La fonction de chiffrement est

$$\begin{array}{rccc} c: & \mathbb{Z}/n\mathbb{Z} & \to & \mathbb{Z}/n\mathbb{Z} \\ & x & \to & ax+b \end{array}$$

Exemple : $A = \{a, b, c, ..., y, z\}$ Bijection de A dans $\mathbb{Z}/26\mathbb{Z}$:

a	b	c	d	...	z
0	1	2	3	...	25

On prend $c(x) = 5x + 10$
Le message *informatique* est chiffré par *yxjcrskbymge*
Le i correspond à la classe 8 ; $c(8) = 5 \times 8 + 10 = 50 = 24$ donc le i est remplacé par y
Le n correspond à la classe 13 ; $c(13) = 5 \times 13 + 10 = 75 = 23$ donc le n est remplacé par x
etc ...

La fonction de déchiffrement est c^{-1} que l'on trouve très facilement : $\left\{ \begin{array}{c} x \in \mathbb{Z}/26\mathbb{Z} \\ y = c(x) = 5x + 10 \\ y \in \mathbb{Z}/26\mathbb{Z} \end{array} \right.$

$y = 5x + 10 \Leftrightarrow y - 10 = 5x \Leftrightarrow 21(y - 10) = 21 \times 5x = 105x = x \Leftrightarrow x = 21y - 210 = 21y + 24$

La classe 5 est inversible dans $\mathbb{Z}/26\mathbb{Z}$ (5 et 26 sont premiers entre eux) et 21 est la classe inverse de 5.
Finalement, la fonction de déchiffrement est $c^{-1}(x) = 21x + 24$
Si l'on applique, par exemple, la fonction de déchiffrement à y (classe 24), on obtient
$c^{-1}(24) = 21 \times 24 + 24 = 528 = 8$ donc y est remplacé par i

Remarque : en prenant $a = 1$, la fonction de chiffrement devient $c(x) = x + b$
C'est alors un simple décalage et l'on retrouve le chiffre de César.

3.2.3 Permutation aléatoire

Les systèmes de chiffrement par substitution mono-alphabétique les plus généraux utilisent **une permutation aléatoire sur l'alphabet**.
Sur un alphabet de 26 lettres, cela donne un nombre de clés possibles considérable, égal à $26! \simeq 4 \times 10^{26}$
Trouver la bonne permutation, en procédant de manière exhaustive à raison d'une permutation testée en 10^{-9} seconde, nécessiterait plus de 10 milliards d'années ...

Exemple : L'alphabet utilisé est $A = \{a, e, i, o, u, n, f, p\}$
Le tableau suivant donne une permutation (donc une bijection) de A dans A : c'est la clé secrète.

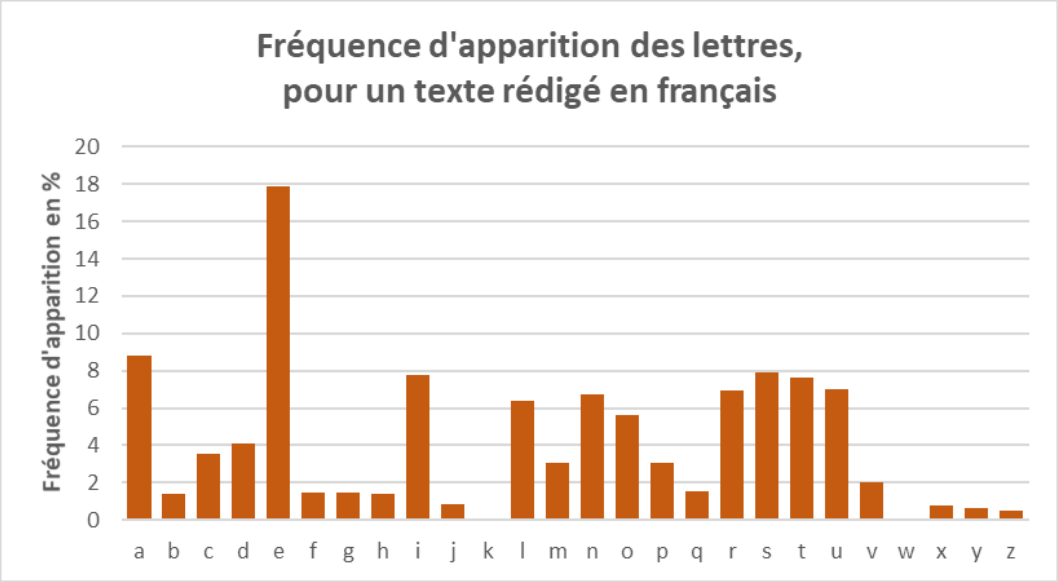
α	a	e	i	o	u	n	f	p
$c(\alpha)$	e	u	n	p	f	i	o	a

Le chiffrement du message *info* est *niop*

3.2.4 Cryptanalyse par analyse des fréquences

Un chiffrement affine est extrêmement vulnérable du fait du faible nombre de clés possibles : une attaque exhaustive (force brute) qui consiste à essayer toutes les clés possibles peut se faire facilement.

Et de manière générale, les chiffrements par substitution mono-alphabétique ne respecte par le principe de diffusion. Avec un tel procédé, une lettre donnée est toujours chiffrée de la même façon. Or, dans la langue française, la lettre e est largement plus utilisée que les autres. Une étude statistique des textes codés permet donc rapidement de déterminer comment est chiffrée le e.



Une attaque par analyse des fréquences permettra donc la cryptanalyse d'un chiffrement par substitution mono-alphabétique.

3.3 Chiffrement par substitution poly-alphabétique

Les chiffrements par substitution poly-alphabétique ont été développés à partir du XVème siècle.
Un chiffrement par substitution poly-alphabétique consiste à remplacer un symbole du texte clair par un autre qui **dépend de l'état du cryptosystème**. Cette modification de la substitution va donc dépendre de la position du symbole dans le message à chiffrer. **Un même symbole du texte clair ne sera pas ainsi systématiquement remplacé par un autre**.
Une cryptanalyse par analyse fréquentielle d'un chiffrement par substitution poly-alphabétique devra utiliser des techniques plus évoluées que pour des substitution mono-alphabétiques.

3.3.1 Chiffrement de Vigenère

Ce chiffrement a déjà été présenté précédemment. Nous reprenons l'exemple proposé.

Exemple : sur $A = \{a, b, c, ..., x, y, z\}$, chiffrons le message *je suis en but informatique*, avec la clé *rodez*.
Nous supprimons les espaces, le message devient *jesuisenbutinformatique*

Nous mettons et répétons la clé autant de fois que nécessaire, en vis à vis du message.
Les lettres de la clé donnent alors le décalage à appliquer pour chiffrer.

Ainsi, le r de *rodez* donne un décalage de 17. Il s'ensuit que le j est décalé de 17 et est donc chiffré en a.
Le o de *rodez* donne un décalage de 14. Il s'ensuit que le e est décalé de 14 et est donc chiffré en s.

Le tableau suivant donne le chiffrement complet.

message	j	e	s	u	i	s	e	n	b	u	t	i	n	f	o	r	m	a	t	i	q	u	e
clé clé clé ...	r	o	d	e	z	r	o	d	e	z	r	o	d	e	z	r	o	d	e	z	r	o	d
chiffrement	a	s	v	y	h	j	s	q	f	t	k	w	q	j	n	i	a	d	x	h	h	i	h

Finalement, le chiffrement obtenu est *asvyhjsqftkwqjniadxhhih*

Cryptanalyse quand la longueur de la clé est connue

Pour un chiffre de Vigenère, la connaissance de la longueur de la clé permet une cryptanalyse. En effet, si la longueur de la clé est connue (notons la l), retrouver le texte clair à partir du texte chiffré peut se faire en appliquant une cryptanalyse par analyse fréquentielle, pour chaque sous-partie du texte chiffré constitué des symboles situés en position de rang congru à une même valeur modulo la longueur l.

Exemple : le texte chiffré suivant a été obtenu par un chiffre de Vigenère avec une clé de longueur 4

kysophjszfgiiygzoyafliapsutqoczzhwgawumbldkdhlzwyuojvcyhbdkghcyebyzituz
hlhjgqcxopjgfsulcyyzxplgwwuxzhgubaumbldkbljawzxkaloxsyfuwuxkhvcvzbmrcuaz
stjyxlggfjbkfhcrszskiezollmyiygkgwytglyyghhyfpytjvcxobxkvvlyghhysunkbklk
obwabilawamkisctqvhtisyjczwuiyvkzlmsophyqyioglyyhycyhlyzzlduiyjuiyguwzyx
ojisalfgbbczxlhkflagfkyxophozvljziziofxoohvghsucrszpuwsyyobfuwuxkgjytrhhz
jllyvhllzloxsakaouxpoylojllgwqyssanxopmafauzctvkiuvuixokhkyncbrbsynkhkyhfbskflyttsyaf

On extrait les 4 sous-parties : chacune de ces sous-parties est chiffrée avec un même décalage.

Sous-partie 1 : position de rang 0 modulo 4
kpziolsohwlhyvbhbtlqpsypwhallzlyuvbutljhzelywlhpvbvhukbiasvszylpylyllyyzjlbllkpvzxvuzsbujhlhlauylqapatuxkbykbls

Sous-partie 2 : position de rang 1 modulo 4
yhfyyiucwudlucdcyuhcjuylugudjxofxcmajgbcszmgyyhycxlhnlwlmchywvmhiycydjgyifchayhliogcpyfxyhllokxllynmuvvoyrnysyy

Sous-partie 3 : position de rang 2 modulo 4
sjggaatzgmkzoykyzzjxglzgxumkakxukvrzygkrkoyktyytxkyykkaakttjuksyoyyzuuuxsgzkgxojoohruyuktzylxapogsxazkuknbkhkta

Sous-partie 4 : position de rang 3 modulo 4
osizfpqzabdwjhgeihgofcxwzbbbwaswhzcsxffsiligggfjovgsbobwiqicizoqghhziiwoabxffozifhsswowgrjvzsoojwsofciihcshfftf

Une cryptanalyse fréquentielle de la première sous-partie donne :
→ 20 lettres l
→ 11 lettres y
→ 9 lettres h
→ 9 lettres b
→ etc ...

Dans cette première sous-partie, la lettre *l* chiffre probablement la lettre *e* ce qui donne un décalage de 7.
Ce décalage de 7 nous donne la première lettre de la clé : **H**
Le déchiffrement de la première sous-partie donne alors :
disbhelhapearouaumejilripateesernounmecasxerpeaiouoandubtlolsreirereerrsceueediosqonsluncaeaetnrejtitmnqdurduel

Nous procédons de même pour chacune des 3 autres sous-parties.

Finalement, le texte clair est

demaindeslaubealheureoublanchitlacampagnejepartiraivoistujesaisquetumattends
jiraiparlaforetjiraiparlamontagnejenepuisdemeurerloindetoipluslongtemps
jemarcherailesyeuxfixessurmespenseessansrienvoiraudehorssansentendreaucunbruit
seulinconnuledoscourbelesmainscroiseestristeetlejourpourmoiseracommelanuit
jeneregarderainilordusoirquitombenilesvoilesauloindescendantversharfleur
etquandjarriveraijemettraisurtatombeunbouquetdehouxvertetdebruyereenfleur

La clé est **HUGO**

Demain, dès l'aube...
Victor Hugo

Demain, dès l'aube, à l'heure où blanchit la campagne,
Je partirai. Vois-tu, je sais que tu m'attends.
J'irai par la forêt, j'irai par la montagne.
Je ne puis demeurer loin de toi plus longtemps.

Je marcherai les yeux fixés sur mes pensées,
Sans rien voir au dehors, sans entendre aucun bruit,
Seul, inconnu, le dos courbé, les mains croisées,
Triste, et le jour pour moi sera comme la nuit.

Je ne regarderai ni l'or du soir qui tombe,
Ni les voiles au loin descendant vers Harfleur,
Et quand j'arriverai, je mettrai sur ta tombe
Un bouquet de houx vert et de bruyère en fleur.

Obtention de la longueur de la clé par test de Kasiski (XIXème siècle)

Le test de Kasiski est une méthode permettant de déterminer la longueur de la clé. Si deux groupes de lettres du texte chiffré sont égaux, alors il est probable qu'ils proviennent d'un même groupe de lettre du texte clair, chiffré avec la même partie de la clé.
On relève alors les écarts entre ces groupes identiques. Ces écarts seront probablement égaux à des multiples de la longueur de la clé.
S'il y a plusieurs répétitions de groupes, le plus grand diviseur commun des distances les séparant est probablement la longueur de la clé.

Appliquons le test de Kasiski à l'exemple précédent. En cherchant les répétitions de chaines de 6 caractères (pour ne pas en avoir trop) dans le texte chiffré, nous obtenons deux motifs qui se répètent à chaque fois deux fois : ***umbldk*** (positions 37 et 105 : distance 68) et ***fuwuxk*** (positions 125 et 345 : distance 220).

*kysophjszfgiiygzoyafliapsutqoczzhwgaw**umbldk**dhlzwyuojvcyhbdkghcyebyzituz*
*hlhjgqcxopjgfsulcyyzxplgwwuxzhguba**umbldk**bljawzxkaloxsy**fuwuxk**hvcvzbmrcuaz*

stjyxlggfjbkfhcrszskiezollmyiygkgwytglyyghhyfpytjvcxobxkvvlyghhysunkbklk
obwabilawamkisctqvhtisyjczwuiyvkzlmsophyqyioglyyhycyhlyzzlduiyjuiyguwzyx
*ojisalfgbbczxlhkflagfkyxophozvljziziofxoohvghsucrszpuwsyyob**fuwuxk**gjytrhhz*
jllyvhllzloxsakaouxpoylojllgwqyssanxopmafauzctvkiuvuixokhkyncbrbsynkhkyhfbskflyttsyaf

Le plus grand diviseur commun de ces distances ($pgcd(68, 220)$) donne 4 : la clé de chiffrement est très probablement de longueur 4.

Obtention de la longueur de la clé par calcul de l'indice de coïncidence de Friedman (1920)

C'est une méthode, plus générale, qui permet la détermination de la longueur de la clé dans un système de chiffrement poly-alphabétique.

3.3.2 Chiffrement de Hill

C'est un système de chiffrement par substitution poly-alphabétique, proposé par Hill en 1931.
On chiffre et on déchiffre par "paquets" de p lettres ce qui rend plus difficile une analyse fréquentielle.

Sur l'alphabet $A = \{a, b, c, ..., x, y, z\}$ chaque lettre est associée à une classe dans $\mathbb{Z}/26\mathbb{Z}$: a est associée à la classe 0, b est associée à la classe 1, ..., z est associée à la classe 25.

La clé de chiffrement est une matrice M, carrée inversible de taille p à coefficients dans $\mathbb{Z}/26\mathbb{Z}$.

Pour chaque paquet de p lettres du texte clair $(m_1, m_2, ...m_p) \in (\mathbb{Z}/26\mathbb{Z})^p$ le chiffrement donne le paquet de p lettres $(c_1, c_2, ...c_p) \in (\mathbb{Z}/26\mathbb{Z})^p$ obtenu par multiplication matricielle :

$$chiffrement\ :\ \begin{pmatrix} c_1 \\ c_2 \\ . \\ . \\ . \\ c_p \end{pmatrix} = M \begin{pmatrix} m_1 \\ m_2 \\ . \\ . \\ . \\ m_p \end{pmatrix} \qquad déchiffrement\ :\ \begin{pmatrix} m_1 \\ m_2 \\ . \\ . \\ . \\ m_p \end{pmatrix} = M^{-1} \begin{pmatrix} c_1 \\ c_2 \\ . \\ . \\ . \\ c_p \end{pmatrix}$$

La fonction de chiffrement est linéaire. Le déchiffrement se fait bien sûr grâce à la matrice inverse M^{-1}

Si la longueur du message à chiffrer n'est pas un multiple de p, alors on complète avec des lettres arbitraires.

Exemple :

Chiffrons le message *je suis en but informatique*, avec un chiffrement de Hill par paquet de 2 lettres.
Nous supprimons les espaces, le message devient *jesuisenbutinformatique*
La longueur du message est 23 : on complète par exemple avec un a pour obtenir une longueur qui sera multiple de 2 : *jesuisenbutinformatiquea*
La matrice de chiffrement choisie est $M = \begin{pmatrix} 13 & 16 \\ 11 & 21 \end{pmatrix}$
On découpe le message à chiffrer par paquet de 2 lettres : *(je)(su)(is)(en)(bu)(ti)(nf)(or)(ma)(ti)(qu)(ea)*
Ces paquets correspondent à chaque fois à un vecteur dans $(\mathbb{Z}/26\mathbb{Z})^2$:
(9,4)(18,20)(8,18)(4,13)(1,20)(19,8)(13,5)(14,17)(12,0)(19,8)(16,20)(4,0)

On chiffre chaque vecteur en multipliant par M. Par exemple : $\begin{pmatrix} 13 & 16 \\ 11 & 21 \end{pmatrix} \begin{pmatrix} 9 \\ 4 \end{pmatrix} = \begin{pmatrix} 25 \\ 1 \end{pmatrix}$

Cela donne les vecteurs suivants dans $(\mathbb{Z}/26\mathbb{Z})^2$:
(25,1)(8,20)(2,24)(0,5)(21,15)(11,13)(15,14)(12,17)(0,2)(11,13)(8,24)(0,18)
Finalement le chiffrement obtenu est : *zbiucyafvplnpomraclniyas*

Le déchiffrement se fait avec la matrice M^{-1}. On a $\det M = 13 \times 21 - 11 \times 16 = 97 = 19$.
La classe inverse de 19 dans $\mathbb{Z}/26\mathbb{Z}$ est 11 ($11 \times 19 = 209 = 1$)
Donc $M^{-1} = 11 \begin{pmatrix} 21 & -16 \\ -11 & 13 \end{pmatrix} = \begin{pmatrix} 23 & 6 \\ 9 & 13 \end{pmatrix}$

Par exemple, si l'on calcule $M^{-1} \begin{pmatrix} 25 \\ 1 \end{pmatrix} = \begin{pmatrix} 23 & 6 \\ 9 & 13 \end{pmatrix} \begin{pmatrix} 25 \\ 1 \end{pmatrix}$, on retrouve bien le vecteur $\begin{pmatrix} 9 \\ 4 \end{pmatrix}$

3.3.3 Chiffrement de Vernam : technique du masque jetable

Nous avons déjà présenté cette technique, mise au point par Gilbert Vernam en 1918. Elle s'inspire du chiffre de Vigenère mais présente quelques caractéristiques essentielles qui en font **un système parfaitement sécurisé**.

Caractéristique du chiffre de Vernam :

- la clé est constituée d'une liste *aléatoire* de nombres compris entre 0 et 25
  (ou bien d'une suite de lettres, chaque lettre correspondant à un nombre entre 0 et 25),
- la clé doit avoir *la même longueur* que le message en clair,
- la clé ne doit être utilisée *qu'une seule fois*.

On parle alors de masque (la clé) jetable (à détruire une fois utilisé). Sous ces conditions, cette technique est parfaitement sécurisée.

A titre d'exemple, chiffrons le message *je suis en but informatique*.
Nous supprimons les espaces, le message devient *jesuisenbutinformatique*
Le masque choisi aléatoirement est *asdeztakkianmpajhyzsyze* ; il a la même longueur que le message en clair.
Nous mettons le masque en vis à vis du message.
Les lettres du masque donnent alors le décalage à appliquer pour chiffrer.

Le tableau suivant donne le chiffrement complet.

message	j	e	s	u	i	s	e	n	b	u	t	i	n	f	o	r	m	a	t	i	q	u	e
clé clé clé ...	a	s	d	e	z	t	a	k	k	i	a	n	m	p	a	j	h	y	z	s	y	z	e
chiffrement	j	w	v	y	h	l	e	x	l	c	t	v	z	u	o	a	t	y	s	a	o	t	i

Finalement, le chiffrement obtenu est *jwvyhlexlctvzuoatysaoti*

Le chiffre de Vernam est en fait utilisé pour **le chiffrement dit à flot**.

3.3.4 Chiffrement à flot de Vernam

L'idée est celle présentée dans le paragraphe précédent mais :
- le message transmis est une suite de symboles binaires (qui codent donc le message à transmettre),
- le masque jetable est une suite *aléatoire* de symboles binaires.

Exemple :

						Chiffrement									
Message	:	$m =$	.	.	.	0	1	1	0	0	0	1	1	.	.	.
Suite chiffrante (aléatoire)	:	$k =$	.	.	.	0	1	0	0	0	1	0	1	1	.	.	.
Message chiffré	:	$c =$	.	.	.	0	0	1	0	0	1	0	0	0	.	.	.
						Déchiffrement									
Message chiffré	:	$c =$	.	.	.	0	0	1	0	0	1	0	0	0	.	.	.
Suite chiffrante (la même)	:	$k =$	.	.	.	0	1	0	0	0	1	0	1	1	.	.	.
Message	:	$m =$	.	.	.	0	1	1	0	0	0	0	1	1	.	.	.

Le chiffrement du bit m_t, à partir du bit aléatoire k_t, repose sur une addition modulo 2 :

$\rightarrow$ **c'est un ou exclusif**

De même, le déchiffrement du bit c_t, à partir du bit aléatoire k_t, repose sur une addition modulo 2.

L'opération de déchiffrement est la même que l'opération de chiffrement. On a :

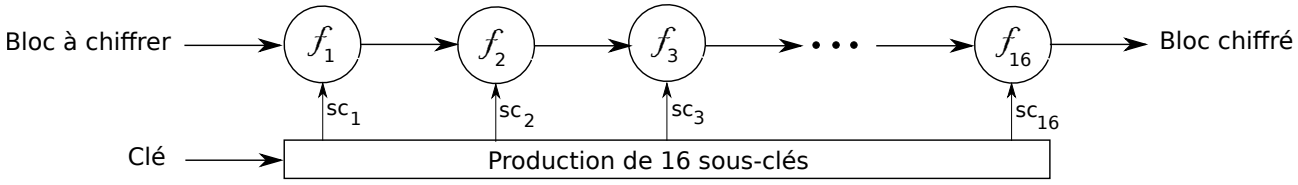
$$m_t \oplus k_t = c_t \qquad et \qquad c_t \oplus k_t = (m_t \oplus k_t) \oplus k_t = m_t \oplus (k_t \oplus k_t) = m_t \oplus 0 = m_t$$

Il faut bien comprendre que le chiffrement de Vernam assure **une sécurité inconditionnelle** : il n'y a aucun moyen de reconstituer le message. Sous réserve, bien sûr, de transmettre la suite chiffrante (le masque jetable) de manière parfaitement sécurisée.

3.4 Le DES : Data Encryption Standard

Pour finir avec cette initiation à la cryptographie symétrique, nous terminons par une présentation rapide du DES, proposé en 1977 par IBM. C'est un procédé de chiffrement par bloc : le message est partagé en bloc de même taille (par exemple 128 bits).
Grâce à une transformation inversible qui dépend de la clé, chaque bloc est transformé en un bloc chiffré.

Le chiffrement d'un bloc repose sur les éléments suivants :
- une fonction de tour, qui opère sur un bloc et qui se fait rapidement,
- la répétition, sur plusieurs tours (16 pour le DES), de ce même processus,
- un dispositif qui, à partir de la clé, produit plusieurs (16 pour le DES) sous-clés qui servent de paramètres à chaque fonction de tour.



4 Initiation à la cryptographie asymétrique

Les algorithmes de chiffrement symétrique nécessitent l'échange préalable, et de manière parfaitement sécurisée, d'une clé. Les algorithmes de chiffrement sont public mais par contre, la clé de chiffrement est évidemment privée et ne doit en aucun cas être divulguée.

Mais comment un émetteur et un destinataire, qui ont besoin d'échanger de manière discrète pour la première fois, peuvent-ils échanger une clé privée sur un réseau ouvert ?

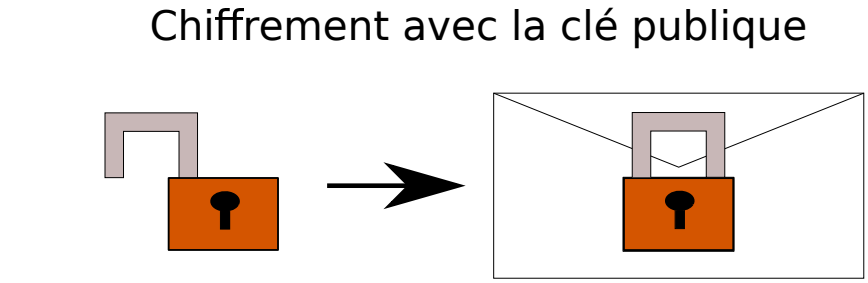
La cryptographie asymétrique, dite aussi à clé publique, va permettre de répondre à cette question.

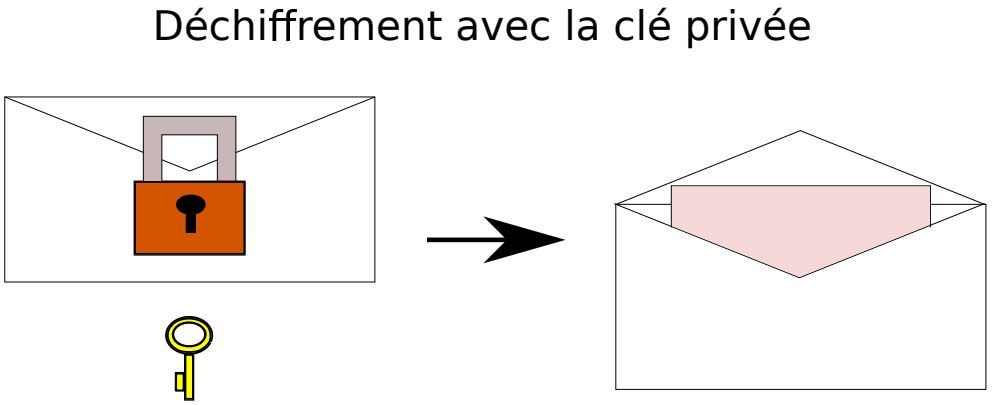
4.1 Principes généraux

Un système de chiffrement asymétrique repose sur deux clés :

- une clé publique, accessible à tous, permettant le chiffrement des messages.
- une clé privée, connue et seulement réservée à la personne ayant donné la clé publique.
  Elle permet le déchiffrement des messages.

On peut faire l'analogie suivante. Les clés publiques sont des cadenas ouvert, que tout le monde peut emprunter. Chiffrer un message revient à fermer un cadenas. Et seule la personne ayant distribué les cadenas est capable de les réouvrir (la clé du cadenas est la clé privée).





4.2 Le chiffrement RSA

Le chiffrement RSA (du nom des inventeurs : Ronald **R**ivest, Adi **S**hamir et Leonard **A**dleman) date de 1977 et est le plus utilisé des procédés de chiffrement à clé publique. Il peut servir, par exemple, à échanger de manière sécurisé une clé de chiffrement symétrique.

4.2.1 Mise en place d'un système RSA

La mise en place d'un système RSA se fait de la façon suivante :

•1• Choisir p et q, 2 nombres premiers distincts.

•2• On note $n = pq$; n est le **module de chiffrement**.

•3• On calcule la fonction d'Euler de n.

$$\varphi(n) = \varphi(p)\ \varphi(q) = (p-1)\ (q-1)$$

•4• Choisir e un entier premier avec $\varphi(n)$. e est l'**exposant de chiffrement**.

•5• Comme $pgcd(e, \varphi(n)) = 1$, e est inversible modulo $\varphi(n)$.
Donc il existe d tel que $ed \equiv 1[\varphi(n)]$. d est l'**exposant de déchiffrement**.

(n, e) est la **clé publique** : accessible à tous.
(n, d) est la **clé privée** : connue seulement de la personne publiant la clé publique.

Ce système RSA permet de coder tous les entiers entre 0 et $n-1$ (on travaille modulo n).

4.2.2 Comment chiffrer un message

En pratique, le message à chiffrer est un entier m ; on suppose que cet entier est inférieur à n.
Si ce n'est pas le cas, on découpe le message m en "mots" plus petits. Donc on peut supposer que $m < n$.
Et on travaille modulo n, donc dans $\mathbb{Z}/n\mathbb{Z}$.

La fonction de chiffrement utilise la clé publique (n, e) ; elle est donnée par :

$$\begin{array}{rccc} f: & \mathbb{Z}/n\mathbb{Z} & \to & \mathbb{Z}/n\mathbb{Z} \\ & m & \to & c = f(m) = m^e \end{array}$$

Le chiffrement consiste à élever l'entier m à la puissance e, modulo n

4.2.3 Comment déchiffrer un message

La fonction f est bijective et f^{-1} (qui utilise la clé privée (n, d)) est donnée par :

$$\begin{array}{rccc} f^{-1}: & \mathbb{Z}/n\mathbb{Z} & \to & \mathbb{Z}/n\mathbb{Z} \\ & c & \to & m = f^{-1}(c) = c^d \end{array}$$

Le déchiffrement consiste à élever l'entier c à la puissance d, modulo n

4.2.4 Un exemple

Reprenons les différentes étapes de la mise en place d'un codage RSA et expérimentons les sur un exemple.

On choisit $p = 13$ et $q = 5$; ainsi, $n = 65$ et $\varphi(n) = 12 \times 4 = 48$.
On choisit un exposant de chiffrement $e = 11$; il nous faut déterminer l'inverse de 11 modulo 48.

$$\begin{gathered}
On\ a\ successivement \\
48 = 11 \times 4 + 4 \\
11 = 4 \times 2 + 3 \\
4 = 3 \times 1 + 1 \\
donc,\ en\ remontant\ l'algorithme\ d'Euclide,\ on\ obtient \\
1 = 4 - 3 \times 1 \\
1 = 4 - (11 - 4 \times 2) \times 1 = 4 \times 3 - 11 \times 1 \\
1 = (48 - 11 \times 4) \times 3 - 11 \times 1 = 48 \times 3 - 11 \times 13 \\
L'inverse\ cherché\ est\ donc\ d = -13 \equiv 35[48]
\end{gathered}$$

En définitive, la clé publique est $(65, 11)$ et la clé privée est $(65, 35)$.

Soit alors le message $m = 20$ à envoyer. On le chiffre en calculant $c = m^{11}[65]$
C'est un calcul d'exponentiation modulaire ; on peut procéder de la manière suivante :

$$\begin{array}{r|c}
m \equiv 20[65] & or\ 11 = 8 + 2 + 1 \\
m^2 \equiv 20^2 \equiv 10[65] & donc\ m^{11} \equiv m^8 \times m^2 \times m^1 \equiv 55 \times 10 \times 20 \equiv 15[65] \\
m^4 \equiv 10^2 \equiv 35[65] & ainsi,\ c = 15 \\
m^8 \equiv 35^2 \equiv 55[65] &
\end{array}$$

Le déchiffrement du message se fait en calculant $c^{35}[65]$. On procède selon le même principe qu'au dessus.

$$\begin{array}{r|c}
c \equiv 15[65] & or\ 35 = 32 + 2 + 1 \\
c^2 \equiv 15^2 \equiv 30[65] & donc\ c^{35} \equiv c^{32} \times c^2 \times c^1 \equiv 35 \times 30 \times 15 \equiv 20[65] \\
c^4 \equiv 30^2 \equiv 55[65] & ainsi,\ c^d \equiv 20 \equiv m[65] \\
c^8 \equiv 55^2 \equiv 35[65] & \\
c^{16} \equiv 35^2 \equiv 55[65] & \\
c^{32} \equiv 55^2 \equiv 35[65] &
\end{array}$$

On retrouve bien le message initial !

4.2.5 Un autre exemple

Nous proposons un deuxième exemple avec deux nombres premiers p et q plus grands. Le système sera (un peu...) plus sécurisé.

Les calculs sont réalisés avec MAPLE -  - et sont présentés page suivante.

Choix d'un premier nombre premier

$p := \mathbf{ithprime}(500)$

3571

Choix d'un deuxième nombre premier

$q := \mathbf{ithprime}(1000)$

7919

Calcul de n : le module de chiffrement

$n := p \cdot q$

28278749

Calcul de la fonction d'Euler en n

$\mathbf{phin} := (p-1)\cdot(q-1)$

28267260

Choix de e : l'exposant de chiffrement

$e := 56761$

56761

On vérifie que e et phin sont premiers entre eux et on récupére l'inverse de e

$\mathbf{igcdex}(e, \mathbf{phin}, 'u', 'v')$

1

Calcul de d : l'exposant de déchiffrement

$d := u \bmod \mathbf{phin}$

2505961

On vérifie que e et d sont inverses l'un de l'autre modulo phin

$e \cdot d \bmod \mathbf{phin}$

1

Choix d'un message à coder (un entier entre 0 et $n-1$)

$M := 6515$

6515

Le codage de M

$\mathbf{MSecret} := M^e \bmod n$

11659718

Le décodage

$\mathbf{MSecret}^d \bmod n$

6515

4.2.6 Preuve du système RSA

Quand on chiffre m et que l'on déchiffre, cela revient à élever m à la puissance e (chiffrement) et ensuite à la puissance d (déchiffrement). On doit donc avoir $(m^e)^d = m^{ed} \equiv m[n]$

Cela repose sur le théorème d'Euler.

4.2.7 Sécurité du système RSA

Tout le monde connaît la clé publique (n, e). Cela permet le chiffrement.

Pour déchiffrer, il faut connaître la clé privée (n, d).

Donc, il faut trouver d à l'aide de n et e.
On rappelle que d est l'inverse de e, modulo $\varphi(n) = (p-1)(q-1)$

Pour déterminer d, il faut donc trouver les deux entiers p et q, autrement dit **factoriser** n.

On rappelle que la multiplication de deux entiers p et q (premiers) est une **fonction à sens unique** :
- il est facile de réaliser cette multiplication : $n = p \times q$,
- **il est par contre très difficile de factoriser n et de retrouver ainsi p et q**

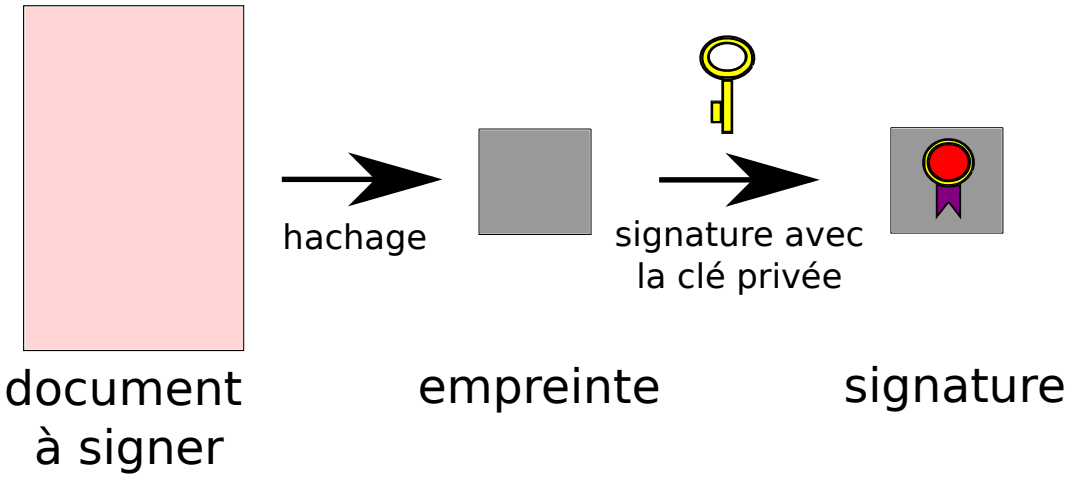
Bien sûr, n doit être un entier très grand pour rendre sa factorisation très difficile, voire impossible. Il est donc essentiel de disposer de très grands nombres premiers (codés a minima sur 1024 bits) pour concevoir un système RSA.

4.3 Signature numérique d'un document

Alice dispose d'un cryptosystème à clé publique, par exemple RSA.
⇒ Elle est la seule à connaître la clé privée (n, d) ; la clé publique (n, e) est connue.

Alice souhaite envoyer à Bob un document $\mathcal{D}$ en l'*authentifiant*.
Elle va, pour ce faire, envoyer *une signature numérique*.

Le processus à suivre est schématisé ci-dessous :



- Dans un premier temps, le document à signer est *haché à l'aide d'une fonction de hachage.*
  → Cela produit *une empreinte.*
- L'empreinte est ensuite signée, et pour cela Alice utilise sa clé privée.
  Notons $\mathcal{E}$ l'empreinte. Alice calcule $\mathcal{E}^d$ *modulo* $n = \widehat{\mathcal{E}}$
- Alice envoie à Bob le document $\mathcal{D}$ et sa signature $\widehat{\mathcal{E}}$

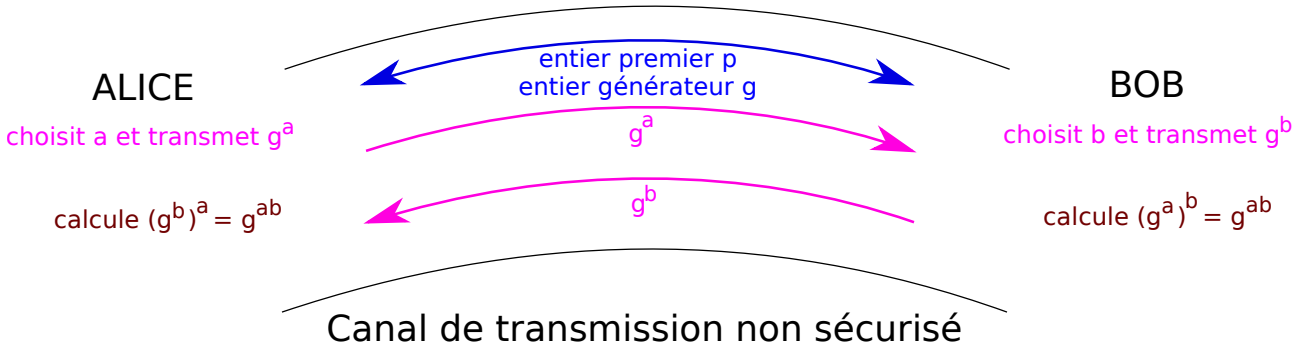
A la réception, Bob va vérifier que le document $\mathcal{D}$ est bien authentique. Pour cela, il soumet le document $\mathcal{D}$ à la fonction de hachage, ce qui donne l'empreinte $\mathcal{E}$. Ensuite, à l'aide de la clé publique (n, e), il calcule $\widehat{\mathcal{E}}^e$ *modulo* n et compare le résultat obtenu avec $\mathcal{E}$. En cas de non égalité, le document n'est pas authentifié.

4.4 Echange de Diffie-Hellman

L'échange de Diffie-Hellman, présenté en 1976, permet à Alice et Bob de partager une donnée secrète en échangeant des informations sur un canal public, non sécurisé. Un observateur qui intercepterait tous les échanges entre Alice et Bob ne pourra pas connaître cette donnée secrète.
La donnée secrète partagée peut ensuite être utilisée comme clé secrète en cryptographie symétrique.

4.4.1 Principe de l'échange

Le schéma ci-dessous présente l'échange de Diffie-Hellman :



Alice et Bob conviennent d'un nombre premier p. Les calculs se feront modulo p, donc dans $\mathbb{Z}/p\mathbb{Z}$.
Ils conviennent également d'un générateur du groupe multiplicatif $((\mathbb{Z}/p\mathbb{Z})^*, \times)$: c'est un entier g compris entre 1 et $p-1$ qui vérifie $(\mathbb{Z}/p\mathbb{Z})^* = \{g, g^2, g^3, ...g^{p-1}\}$

Ces deux entiers, p et g, peuvent être échangés publiquement sans inconvénient.

Alice choisit alors un entier a et transmet g^a à Bob.
Bob, de son côté, choisit un entier b et transmet g^b à Alice.

Alice, qui a reçu g^b, élève à la puissance a et obtient g^{ab}
Bob, qui a reçu g^a, élève à la puissance b et obtient g^{ab}

$\Longrightarrow$ g^{ab} **est une donnée secrète qu'Alice et Bob partagent ensemble.**

4.4.2 Comment trouver cette donnée secrète ?

Un observateur qui souhaiterait trouver la donnée secrète g^{ab} dispose des entiers p, g, g^a et g^b.

Pour trouver g^{ab}, il cherchera par exemple à mettre g^b à la puissance a. Pour cela, il lui faut trouver a.
Trouver a connaissant p, g et g^a est le problème du logarithme discret ; ce problème est difficile et, sous réserve de choisir un entier premier p suffisamment grand, ce problème est actuellement impossible.

4.4.3 Un exemple

Exemple : Alice et Bob conviennent de prendre $p = 179$ et $g = 18$.
$g = 18$ est effectivement un générateur du groupe $(\mathbb{Z}/179\mathbb{Z})^*$
La suite des g^i, pour i de 1 à 178 donne :

$18, 145, 104, 82, 44, 76, 115, 101, 28, 146, 122, 48, 148, 158, 159, 177, 143, 68, 150, 15, 91, 27, 128, 156, 123, 66, 114, 83, 62, 42, 40, 4, 72, 43, 58, 149, 176, 125, 102, 46, 112, 47, 130, 13, 55, 95, 99, 171, 35, 93, 63, 60, 6, 108, 154, 87, 134, 85, 98, 153, 69, 168, 160, 16, 109, 172, 53, 59, 167, 142, 50, 5, 90, 9, 162, 52, 41, 22, 38, 147, 140, 14, 73, 61, 24, 74, 79, 169, 178, 161, 34, 75, 97, 135, 103, 64, 78, 151, 33, 57, 131, 31, 21, 20, 2, 36, 111, 29, 164, 88, 152, 51, 23, 56, 113, 65, 96, 117, 137, 139, 175, 107, 136, 121, 30, 3, 54, 77, 133, 67, 132, 49, 166, 124, 84, 80, 8, 144, 86, 116, 119, 173, 71, 25, 92, 45, 94, 81, 26, 110, 11, 19, 163, 70, 7, 126, 120, 12, 37, 129, 174, 89, 170, 17, 127, 138, 157, 141, 32, 39, 165, 106, 118, 155, 105, 100, 10, 1$

Alice choisit $a = 43$; elle calcule $18^{43} = 130$ qu'elle transmet à Bob.
Bob choisit $b = 81$; il calcule $18^{81} = 140$ qu'il transmet à Alice.

Alice, qui a reçu 140, élève à la puissance a et obtient $g^{ab} = 140^{43} = 131$
Bob, qui a reçu 130, élève à la puissance b et obtient $g^{ab} = 130^{81} = 131$

$\Longrightarrow$ L'entier 131 est un secret partagé entre Alice et Bob.

4.5 Procédé d'El Gamal

Datant de 1985, le chiffrement d'El Gamal (du nom de son inventeur : Taher ElGamal), repose sur l'échange de Diffie-Hellman.

4.5.1 Principe

Alice souhaite envoyer un message m à Bob.

Ils conviennent d'un nombre premier p. Les calculs se feront modulo p, donc dans $\mathbb{Z}/p\mathbb{Z}$.
Ils conviennent également d'un générateur du groupe multiplicatif $((\mathbb{Z}/p\mathbb{Z})^*, \times)$; notons le g.
Enfin, ils conviennent d'une méthode de chiffrement symétrique E pour laquelle la clé secrète est un entier compris entre 1 et $p - 1$.

Par ailleurs, Bob dispose d'une clé privée s (un entier entre 1 et $p - 1$)
La clé publique associée est $y = g^s$ *modulo* p
Nous rappelons qu'il est impossible de retrouver s à partir de y (problème du logarithme discret).

Les deux entiers p et g ainsi que la méthode de chiffrement E peuvent être échangés publiquement.
L'entier y (la clé publique) est également connue.

Chiffrement du message m

Alice choisit un entier a entre 1 et $p-1$
Alice calcule $k = y^a \ modulo \ p$ k est la clé de session
Alice se sert de l'algorithme de chiffrement E, avec la clé secrète k, pour chiffrer m
$\Rightarrow$ elle envoie $c = E_k(m)$ à Bob
Alice calcule $z = g^a \ modulo \ p$
$\Rightarrow$ elle envoie z à Bob

Déchiffrement du message c

Bob a reçu z et c : z va permettre de trouver la clé de session et c contient l'information chiffrée.
Bob calcule la clé de session
$\Rightarrow$ il calcule $z^s = (g^a)^s = (g^s)^a = y^a = k$
Bob utilise ensuite l'algorithme de déchiffrement E^{-1}, avec la clé k, pour retrouver m à partir de c.

4.5.2 Un exemple

Exemple : Alice et Bob conviennent de prendre $p = 179$ et $g = 18$
La méthode de chiffrement symétrique E sera (pour cet exemple très simple), un décalage sur l'alphabet $A = \{a, b, ..., z\}$. La clé secrète est ici la valeur du décalage à appliquer pour chiffrer.

La clé privée de Bob sera $s = 34$
Bob rend donc publique la clé $y = g^s \ modulo \ p = 18^{34} \ modulo \ 179 = 43$

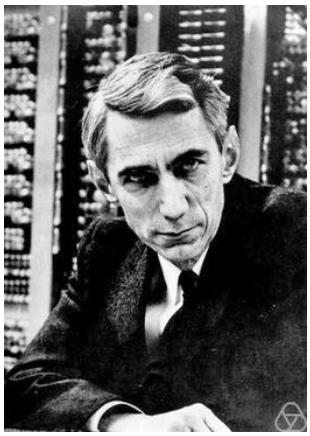
Alice souhaite chiffrer le message $m =$*bonjour*
Elle choisit $a = 67$
Alice calcule la clé de session $k = y^a \ modulo \ p = 43^{67} \ modulo \ 179 = 173$
Elle se sert de cette clé de session pour chiffrer $m =$*bonjour*
Le décalage est $k = 173$ ce qui donne, modulo 26, 17.
Donc $c =$*sfeafli*
Alice calcule $z = g^a \ modulo \ p = 18^{67} \ modulo \ 179 = 53$

$\Rightarrow$ Alice envoie à Bob $c =$*sfeafli* et $z = 53$

A partir de $z = 53$ Bob retrouve la clé de session.
Il calcule $z^s \ modulo \ p = 53^{34} \ modulo \ 179 = 173$ et retrouve k
Bob effectue ensuite un décalage de 173 (donc 17 modulo 26), en sens inverse, pour retrouver *bonjour* à partir de *sfeafli*

5 Codes détecteurs et correcteurs

Nous abordons dans ce chapitre un sujet essentiel : comment assurer l'intégrité dans la transmission des données ? Ce chapitre est une initiation aux codes détecteurs et correcteurs.



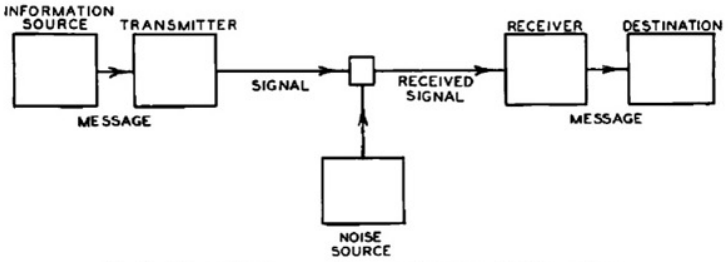


Fig. 1—Schematic diagram of a general communication system.

Schéma originel de Shannon : transmission d'un message

Claude Shannon (1916-2001) : fondateur de la théorie de l'information
Auteur de *La théorie mathématique de la communication (1949)*

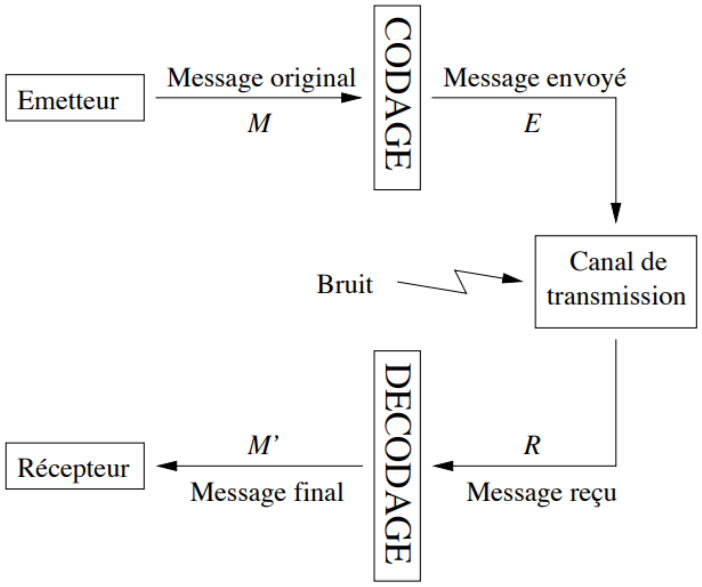
Pour communiquer des données, le codage de l'information se fait habituellement à l'aide d'une suite de 0 et de 1 qui sont transmis sous forme de courant électrique.
On conçoit qu'une telle transmission, si elle se prolonge, contiendra **inévitablement** des erreurs : des 0 sont remplacés par des 1 et des 1 sont remplacés par des 0. Et une erreur d'un seul bit peut avoir des conséquences fâcheuses.

Pour assurer l'intégrité des données, deux orientations sont possibles :
- agir sur le canal de transmission, l'objectif étant de réduire le taux d'erreur.
  Mais de toute façon le taux d'erreur ne sera jamais nul !
- agir sur le message transmis, c'est le principe des codes détecteurs et correcteurs d'erreurs.
  Et c'est l'objet de ce chapitre.

En pratique, on ne se contentera pas d'envoyer uniquement le message à transmettre : on y ajoute une part de **redondance**.
On pourrait par exemple, tout simplement, recommencer la transmission des données. Mais en pratique, cela n'est pas concevable car :
- le message originel n'est plus forcément disponible,
- le temps de transmission peut être trop long,
- une nouvelle transmission contiendra aussi, inévitablement, des erreurs !

Conclusion : quand une erreur est détectée, il faut chercher à la corriger directement, au fur et à mesure de la transmission.

5.1 Schéma général d'une transmission de données



Le message à transmettre M est d'abord **codé** en un message E : introduction d'une part de redondance.
Le message E est envoyé sur le canal de transmission.
Le canal étant imparfait, le message reçu R n'est pas nécessairement identique au message envoyé E : il peut contenir des erreurs.
Le message reçu est **décodé** : des erreurs éventuelles peuvent être détectées et corrigées.
Le message décodé M' est identique au message initial M ... s'il n'y a pas eu trop d'erreur(s) dans la transmission !

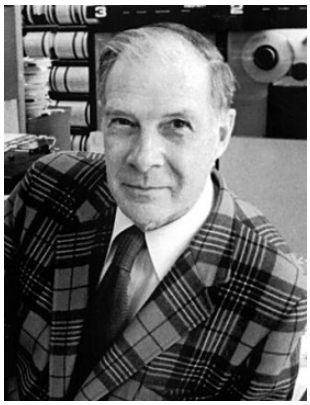
5.2 L'idée de base des codes détecteurs et correcteurs

Pour détecter, puis corriger les erreurs, l'idée est de transmettre une succession de bits qui présente **une certaine particularité**, une certaine propriété.
Ainsi, si je reçois une succession de bits dans laquelle je ne retrouve pas la particularité attendue, j'aurai sans doute détecter une erreur de transmission et peut-être saurai-je la corriger.

Exemple 1 : je reçois le mot binaire 000110111111000
Sans indication particulière, je n'ai pas de raison de suspecter une erreur dans ce message.

Exemple 2 : je reçois le mot binaire 000110111111000
Mais là je sais que je dois recevoir un mot où les bits doivent se suivre par paquets de trois bits identiques.
J'ai donc la certitude qu'au moins une erreur de transmission a eu lieu : détection d'une erreur !
Il reste à la (ou les) corriger ... sans doute en remplaçant le sixième bit par un 1.

5.3 Notion de distance de Hamming



Richard Hamming (1915-1998)

→ travaux sur la détection des erreurs et la correction

En 1946, il travaille sur un calculateur à carte perforée de faible fiabilité.

Durant la semaine, les ingénieurs pouvaient corriger les erreurs ...
... mais pas pendant les périodes chômées, et là les machines s'arrêtent invariablement sur des erreurs.

Cette situation conduit Richard Hamming à inventer le premier code correcteur efficace.

5.3.1 Mot binaire

Nous noterons $\mathbb{B} = \{0, 1\}$ et nous nommerons **mot binaire** une suite de 0 et de 1. La longueur d'un tel mot sera naturellement égale à son nombre de bits. Autrement dit, un mot binaire de longueur n est un élément de $\mathbb{B}^n$. Le poids d'un mot binaire est égal au nombre de 1 qu'il contient.
Par exemple 0100110 est un mot binaire de longueur 7. Son poids est égal à 3.

5.3.2 Addition de deux mots binaires

Définition : Dans $\mathbb{B}$ on définit une addition, notée $\oplus$ par :

$\oplus$	0	1
0	0	1
1	1	0

Cette addition (c'est un *ou exclusif*) vérifie les propriétés suivantes :
- elle est associative → $a \oplus (b \oplus c) = (a \oplus b) \oplus c$
- elle possède un élément neutre → $a \oplus 0 = 0 \oplus a = a$
- tout élément a admet comme symétrique a → $a \oplus a = 0$
- elle est commutative → $a \oplus b = b \oplus a$

Remarques :
- on retrouve ici une situation semblable à l'addition dans $\mathbb{Z}/2\mathbb{Z}$
- chacune des trois égalités suivantes implique les deux autres

$$a \oplus b = c \qquad b \oplus c = a \qquad c \oplus a = b$$

A partir de cette addition, nous définissons maintenant l'addition de deux mots binaires.

Exemple :
$$\begin{array}{rr} & 01001110 \\ \oplus & 10011000 \\ \hline = & 11010110 \end{array} \qquad 01001110 \oplus 10011000 = 11010110$$

Les propriétés vues pour l'addition dans $\mathbb{B}$ restent vraies pour l'addition de deux mots binaires.

5.3.3 Distance de Hamming

Définition : Soient a et b deux mots binaires de $\mathbb{B}^n$
On appelle **distance de Hamming** entre a et b et on note $d(a,b)$ le nombre de position(s) où a et b ont des bits différents : $d(a,b) = card\{i \ tel \ que \ a_i \neq b_i\}$

Vu autrement, c'est aussi le nombre de bits à modifier pour passer de a à b.

Exemple : $d(11000, 10110) = 3$

On a bien défini ici une **distance** sur $\mathbb{B}^n$; comme toute distance, elle vérifie les propriétés suivantes.

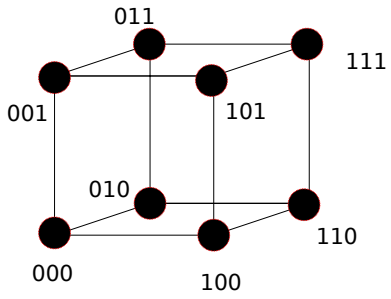
Propriétés : a, b et c sont trois mots binaires appartenant à $\mathbb{B}^n$
- $d(a,b) \geq 0$ (positivité)
- $d(a,b) = 0 \quad \Leftrightarrow \quad a = b$
- $d(a,b) = d(b,a)$ (symétrie)
- $d(a,b) \leq d(a,c) + d(c,b)$ (inégalité triangulaire)

Théorème : On dispose les éléments de $\mathbb{B}^n$ sur un n-cube : un carré si $n = 2$, un cube si $n = 3$, ...
La distance $d(a,b)$ est alors le nombre d'arêtes qu'il faut longer pour se rendre de a à b par le chemin le plus court possible.

Exemple :

Dans $\mathbb{B}^3$ on a :

$d(000, 100) = 1$
$d(000, 111) = 3$
...



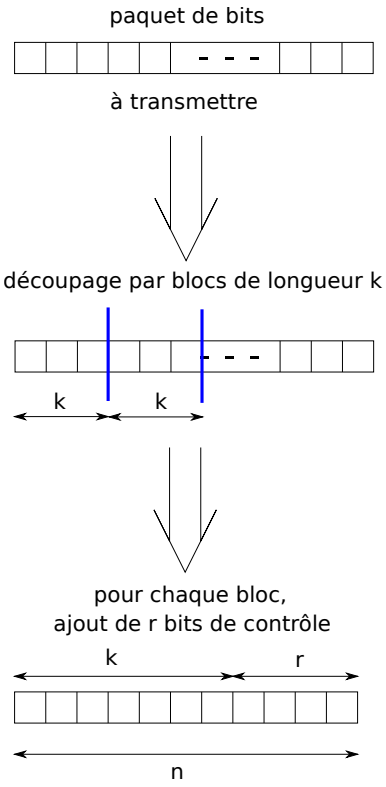
5.4 Codage par bloc

Dans la suite, on se placera dans le cadre d'un **codage par bloc**. C'est une technique très répandue pour la transmission des bits.

Le principe consiste, pour l'émetteur, à :
- découper le paquet de bits à transmettre en blocs, tous de même taille,
- coder chaque bloc en lui ajoutant un nombre fixé de bits de contrôle (c'est la redondance) ; on forme ainsi un mot de code,
- transmettre ce mot de code.

Le receveur :
- vérifie que le mot reçu est bien un mot de code (**détection**), le corrige éventuellement (**correction**),
- extrait le bloc du mot de code qui a été accepté ou corrigé.

Le principe d'un codage par bloc est :



k bits d'informations

+

r bits de contrôle
(ou bits de parité)
qui dépendent du bloc

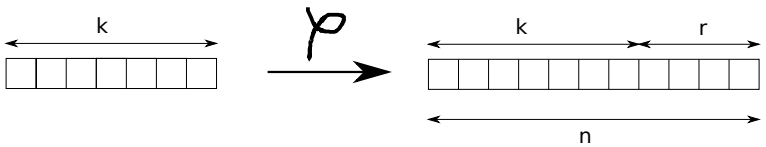
=

n bits formant un mot de code

L'ensemble des mots de code est **le code**.
n est **la longueur** du code.
Notation d'un tel code : $[n, k]$

$\tau = \dfrac{k}{n}$ mesure, parmi les n bits transmis, la proportion de bits porteurs d'information.
C'est **le rendement** (ou le taux) du code.

Le codage est l'application φ de $\mathbb{B}^k$ dans $\mathbb{B}^n$ qui à un bloc associe son mot de code. $\varphi : \mathbb{B}^k \to \mathbb{B}^n$

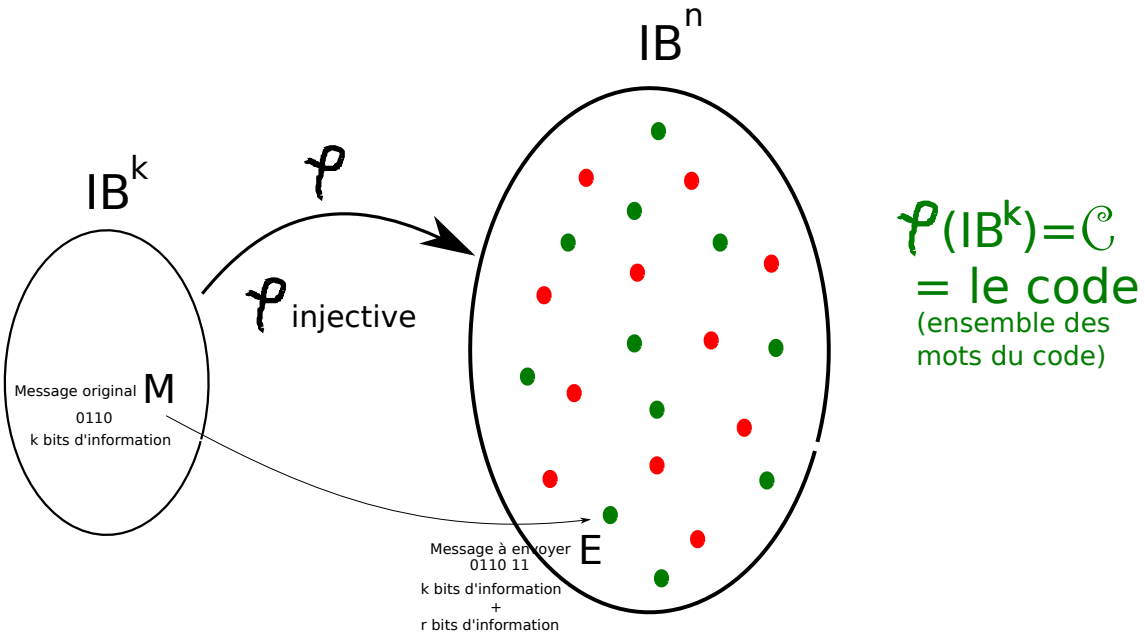


Un codage est dit **systématique** si le mot à coder se retrouve en tête du mot codé.
Dans la suite, nous ne considérerons que des codage systématique.
Cela assure l'injectivité de l'application de codage φ.

Exemple : 01001 est le mot à coder / **01001**0 est le mot codé ($k = 5$, $r = 1$, $n = 6$)
On a ajouté un bit pour contrôler la parité.

5.5 Le codage, la transmission, le décodage : principes généraux

5.5.1 Le codage



5.5.2 La transmission



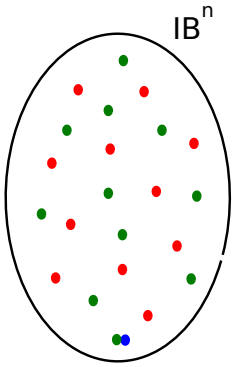
Des erreurs peuvent se produire !

0,1,2,3, ... erreurs ...

N est le nombre d'erreur(s) dans la transmission.

5.5.3 Le décodage

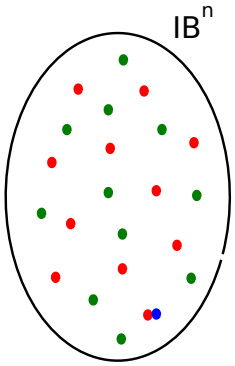
Premier cas : R est un mot du code ($R \in \mathcal{C}$)



Dans ce cas, le plus probable est qu'aucune erreur ne se soit produite lors de la transmission.

$\Rightarrow$ ON NE CORRIGE RIEN ET ON GARDE LE MESSAGE RECU.
(mais peut-être que l'on aura $M' \neq M$)

Deuxième cas : R n'est pas un mot du code ($R \notin \mathcal{C}$)

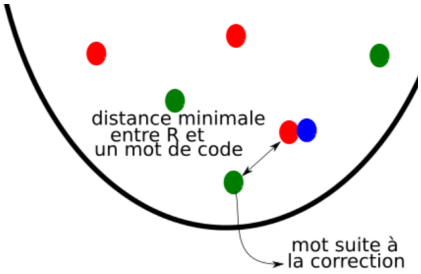


N (nombre d'erreur(s))	Possible ?
0	IMPOSSIBLE
1	POSSIBLE
2	POSSIBLE
3	POSSIBLE
...	...

Au moins une erreur s'est produite
dans la transmission.

$\Rightarrow$ ON DETECTE (au moins) UNE ERREUR.
(il faut donc corriger)

5.5.4 Principe de la correction

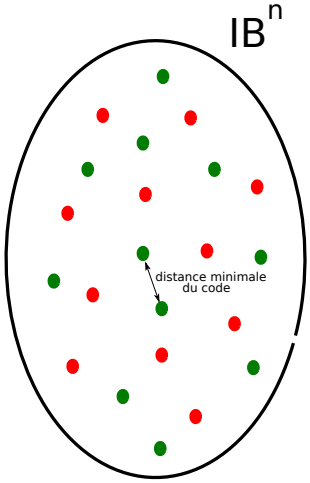


On va chercher le mot du code
le plus proche du mot reçu.

(Utilisation de la distance de Hamming.)

5.6 Distance minimale d'un code

5.6.1 Définition



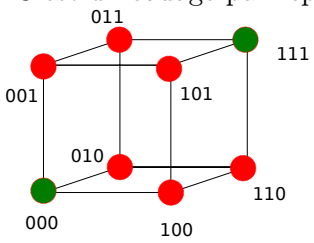
Définition : on note d **la distance minimale** entre 2 mots (différents) quelconques du code.

Autrement dit, $\forall a \in \mathcal{C}$ et $\forall b \in \mathcal{C}$
si $a \neq b$ on a $d(a,b) \geq d$
(2 mots distincts du code sont toujours situés
à une distance $\geq d$; et parfois égale à d)

Evidemment $d \geq 1$

On note un tel code $[n, k, d]$

Exemple : On découpe par bloc de 1 bit et le codage (l'application φ) consiste à ajouter deux fois le même bit. C'est un codage par répétition. L'ensemble des mots de code est $\mathcal{C} = \{000, 111\}$



La distance minimale de ce code est $d = 3$

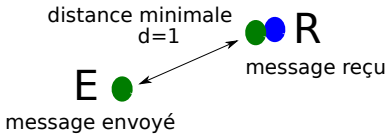
Plus la distance minimale d'un code est grande, meilleur cela est !

5.6.2 Que se passe-t-il s'il y a $N = 1$ erreur lors de la transmission

Quand le message reçu R est différent du message envoyé E, le plus probable est qu'une seule erreur se soit produite lors de la transmission. Il est donc intéressant de considérer cette situation.

Avec une distance minimale $d = 1$

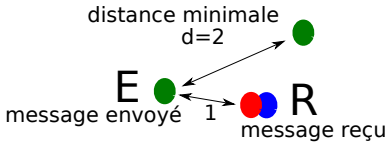
Un message E a été envoyé dans le canal.
R est le message reçu ; une erreur de transmission s'est produite.
R peut donc être un mot de code différent de E.



⇒ LA DETECTION SYSTEMATIQUE D'UNE ERREUR EST IMPOSSIBLE.

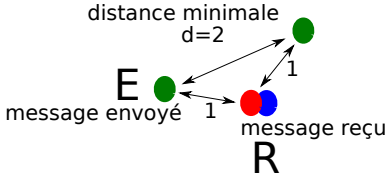
Avec une distance minimale $d = 2$

Un message E a été envoyé dans le canal.
R est le message reçu ; une erreur de transmission s'est produite.
R ne peut donc pas être un mot de code.



⇒ DETECTION SYSTEMATIQUE D'UNE ERREUR.

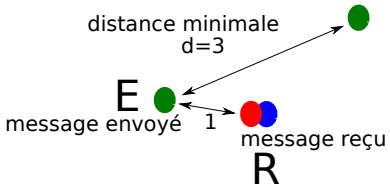
Un message E a été envoyé dans le canal.
R est le message reçu ; une erreur de transmission s'est produite.
R peut être à égale distance de E et d'un autre mot du code.



⇒ LA CORRECTION N'EST PAS SYSTEMATIQUE.

Avec une distance minimale $d = 3$

Un message E a été envoyé dans le canal.
R est le message reçu ; une erreur de transmission s'est produite.
R ne peut donc pas être un mot de code.
Le mot de code le plus proche de R est forcément E.



⇒ DETECTION SYSTEMATIQUE D'UNE ERREUR.
⇒ CORRECTION SYSTEMATIQUE.

5.6.3 Cas général

Considérons un code $[n, k, d]$
→ n est la longueur du code
→ k est le nombre de bits d'informations ($r = n - k$ est le nombre de bits de contrôle)
→ d est la distance minimale du code

Théorème :
- Le receveur détecte de façon certaine tous les messages faux
  tant que le nombre d'erreur(s) N vérifie $0 < N < d$
- Le receveur corrige correctement
  tant que le nombre d'erreur(s) N vérifie $0 \leq N < d/2$

5.6.4 Qualité d'un bon code

Avec ce que nous venons de voir, un code dont la distance minimale est $d = 3$ permet de détecter et de corriger systématiquement $N = 1$ erreur.

Un code de distance minimale $d = 5$ permettrait de détecter et de corriger systématiquement $N = 2$ erreurs.

Evidemment, augmenter la distance minimale du code implique d'augmenter le nombre de bits de parité (donc augmenter la valeur de r). Et ainsi, le rendement du code (et donc la vitesse de transmission) s'affaiblit.

$$d \nearrow \quad \Rightarrow \quad r \nearrow \quad \Leftrightarrow \quad \textit{rendement} \searrow$$

Un bon compromis est de prendre, par exemple, un code dont la distance minimale est $d = 3$, et de faire en sorte que son rendement soit le plus élevé possible.

On pourrait aussi se demander, pour des valeurs de n, k et r fixées, quel pourrait être le maximum de la distance minimale du code. Le paragraphe suivant va répondre à cette question.

5.6.5 Borne de Hamming

Considérons un code $[n, k, d]$
→ n est la longueur du code
→ k est le nombre de bits d'informations ($r = n - k$ est le nombre de bits de contrôle)
→ d est la distance minimale du code

Nous avons vu que :
- si $d = 1$ ou $d = 2$, alors le code ne peut corriger aucune erreur ; nous noterons $t = 0$
- si $d = 3$ (ou si $d = 4$), alors le code corrigera systématiquement $t = 1$ erreur
- si $d = 5$ (ou si $d = 6$), alors le code corrigera systématiquement $t = 2$ erreurs
- etc ...

L'entier t sera le nombre d'erreur(s) que le code peut corriger de manière certaine.

Théorème : inégalité de Hamming

Les entiers n, r et t sont liés par l'**inégalité de Hamming** :

$$C_n^0 + C_n^1 + C_n^2 + ... + C_n^t \leq 2^r$$

Exemples :

•1• Soit un code de longueur $n = 5$ avec $r = 3$ bits de contrôle (seulement 2 bits d'informations).
Combien d'erreur(s) (au maximum) ce code est-il susceptible de corriger ?
On a $C_5^0 = 1$, $C_5^1 = 5$, $C_5^2 = 10$, ... Et d'autre part, $2^3 = 8$
Donc au maximum $t = 1$
⇒ Ce code ne peut pas corriger plus d'une erreur.

•2• Soit un code de longueur $n = 7$ avec $r = 3$ bits de contrôle ($k = 4$ bits d'informations).
On a $C_7^0 = 1$, $C_7^1 = 7$, $C_7^2 = 21$, ... Et d'autre part, $2^3 = 8$
Donc au maximum $t = 1$
⇒ Un tel code ne peut pas corriger plus d'une erreur.
Par contre son rendement est meilleur que celui de l'exemple 1.

Définition : Un code est dit **parfait** quand l'inégalité de Hamming est une égalité.

Exemples : Un code $[n = 3, k = 1, d = 3]$ ($r = 2$) corrigera $t = 1$ erreur. C'est un code parfait ($C_3^0 + C_3^1 = 2^2$)
Un code $[n = 7, k = 4, d = 3]$ ($r = 3$) corrigera $t = 1$ erreur. C'est un code parfait ($C_7^0 + C_7^1 = 2^3$)

6 Codages linéaires

Les codages linéaires sont particulièrement intéressants : ils permettent de coder, de détecter des erreurs et de les corriger très facilement et rapidement. Ils s'appuient sur l'utilisation de l'algèbre linéaire et des matrices.

6.1 Définition et représentation matricielle

6.1.1 Définition et propriétés

Définition : On dit qu'un code $\mathcal{C}$ est **linéaire** quand la somme de deux mots de codes quelconques est encore un mot de code.
Dans ce cas, et c'est équivalent, le codage (donc la fonction φ) est **linéaire**.

Propriété : Quand un code est linéaire, le message 0 est toujours un mot de code. C'est le mot du code associé au bloc nul (constitué uniquement de 0).

Théorème : Quand un code est linéaire :
- La distance minimale du code (donc d) est le poids du mot de code non nul de plus petit poids.
- Quel que soit le mot du code, la distance la plus courte qui sépare ce mot d'un autre mot du code est égale à d.

Conséquence : Dans un code linéaire, **les mots de code sont répartis de façon homogène**.
La répartition autour des mots de code est toujours la même.

6.1.2 Intérêt d'un codage linéaire

Si le codage est linéaire (et systématique), alors la fonction de codage φ est une application linéaire.

Pour connaître la fonction de codage φ (et donc pour coder n'importe quel bloc), **il suffit de connaître comment coder les k blocs suivants** :

- $b_1 = 100...00$
- $b_2 = 010...00$
- $b_3 = 001...00$
- ...
- $b_{k-1} = 000...10$
- $b_k = 000...01$

Ces blocs b_i forment la base canonique de $\mathbb{B}^k$.
Pour coder n'importe quel bloc b, il suffit alors d'ajouter les mots de code des blocs b_j dont les indices sont les indices des bits égaux à 1 dans b.

Exemple : On prend $k = 3$. Alors $b_1 = 100$, $b_2 = 010$ et $b_3 = 001$
On se donne alors le codage linéaire φ suivant :

- $\varphi(b_1) = 1001$
- $\varphi(b_2) = 0101$
- $\varphi(b_3) = 0011$

Ce codage consiste à ajouter à chaque bloc un bit de parité.
Remarquons que le codage est bien systématique : on retrouve le bloc codé en tête du mot de code.

Pour coder le bloc $b = 110$ il suffit alors d'ajouter $\varphi(b_1)$ et $\varphi(b_2)$. Cela donne $\varphi(b) = 1001 + 0101 = 1100$

En pratique, cela va se faire directement et simplement en utilisant des matrices.

6.1.3 Représentation matricielle d'un codage linéaire

Avec un codage linéaire, le calcul matriciel permet très facilement de déterminer le mot de code associé à un bloc.

Définition : Soit $[n, k]$ un codage linéaire systématique.
La **matrice génératrice du codage**, notée G, est une matrice de taille k lignes et n colonnes. Pour la former, on range en lignes les blocs b_i auxquels on ajoute les r bits de contrôle associés à ces blocs.
Elle a la forme suivante : identité de taille k à gauche, **matrice de parité** à droite

$$G = \left(\begin{array}{c|c} & \\ I_k & P \\ & \end{array}\right) \qquad I_k \textit{ identité taille } k \qquad P \in \mathcal{M}_{k,r}(\mathbb{B}) \textit{ matrice de parité}$$

Pour trouver le codage du bloc $b \in \mathbb{B}^k$, on forme alors le produit $b\ G$
et on trouve un mot du code dans $\mathbb{B}^n$.

Retour à l'exemple :
La matrice génératrice du code est $G = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix}$ La matrice de parité est $P = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$

Pour coder le bloc $b = 110$, on forme $b\ G = (110) \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix} = (1100)$

Pour ce code, on peut aussi dire que sa distance minimale est égale à $d = 2$: c'est le poids du mot de code non nul de plus petit poids.
Et si l'on se donne un mot de code quelconque, la plus courte distance qui le sépare d'un autre mot du code est égale à $d = 2$.

Remarque : Pour définir un codage linéaire $[n, k]$, il suffit donc de se donner la matrice de parité P. Et se donner la matrice P, revient à donner $k \times r$ coefficients.
Un codage linéaire $[n, k]$ est donc complètement défini par la donnée de $k \times r$ bits.

6.2 Matrice de contrôle et syndrome d'un message

6.2.1 Matrice de contrôle

Définition : Soit $[n, k]$ un codage linéaire dont la matrice de parité est P.
La **matrice de contrôle du codage**, notée H, est la matrice de taille r lignes et n colonnes formée de la manière suivante :

$$H = \begin{pmatrix} & P^t & & I_r & \end{pmatrix} \qquad On\ a\ aussi \quad H^t = \begin{pmatrix} & P & \\ & & \\ & I_r & \end{pmatrix}$$

Retour à l'exemple :

Pour ce codage, la matrice de parité est $P = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$

La matrice de contrôle est $H = \begin{pmatrix} 1 & 1 & 1 & 1 \end{pmatrix}$ et on a $H^t = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}$

6.2.2 Syndrome d'un message

Définition : Le **syndrome d'un message** m est obtenu en faisant le produit $m\ H^t$

Un syndrome est un mot binaire de longueur r ($\in \mathbb{B}^r$) ; au total, il y a 2^r syndromes possibles.

Retour à l'exemple : Pour ce codage, on a $H^t = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}$

Pour chaque message possible, le tableau ci-contre donne son syndrome :

Remarque : Le syndrome d'un message est nul si le message est un mot du code.

Message	Syndrome
0000	0
0001	1
0010	1
0011	0
0100	1
0101	0
0110	0
0111	1
1000	1
1001	0
1010	0
1011	1
1100	0
1101	1
1110	1
1111	0

6.3 Détection et correction d'un message par syndrome

Le calcul du syndrome d'un message va permettre de détecter une erreur ... mais aussi de corriger.

6.3.1 Détection d'une erreur

Théorème :
Un message n'est pas un mot du code si et seulement si son syndrome n'est pas nul.

Ainsi, si le syndrome d'un message n'est pas nul, c'est qu'il y a forcément eu une ou des erreur(s) lors de la transmission.

Exemple : soit le code $[7, 4]$ défini par sa matrice génératrice.

$$G = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{pmatrix} \qquad P = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}$$

Au total, il y a $2^4 = 16$ mots de code ; ils s'obtiennent en prenant une suite b de 4 bits et en formant le produit $b\,G$.
Par exemple, avec $b = 0110$ on obtient $m = b\,G = 0110001$ qui est un mot du code.

Pour ce codage, la matrice de contrôle est :

$$H = \begin{pmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix} \qquad \textit{et on a} \qquad H^t = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Avec $m = 0110001$, le calcul du syndrome donne $m\ H^t = (000)$: on a affaire à un mot du code.
Mais avec $m = 0110011$, le calcul du syndrome donne $m\ H^t = (010)$: on n'a pas affaire à un mot du code.

Pour ce codage, tous les mots du code sont donnés ci-dessous :

0000000	0001011	0010110	0011101
0100111	0101100	0110001	0111010
1000101	1001110	1010011	1011000
1100010	1101001	1110100	1111111

Le syndrome de ces mots est toujours nul.

6.3.2 Correction d'un message par syndrome

Construction de la liste des syndromes :

Pour corriger un message par syndrome, on commence par former un tableau à deux colonnes : syndrome et message.
Dans la colonne syndrome, on place tous les mots binaires de longueur r dans l'ordre.
Par ailleurs, on prend les mots binaires de $\mathbb{B}^n$, dans l'ordre et par poids croissant. Pour $m \in \mathbb{B}^n$ pris dans cette liste, on calcule son syndrome. On place alors ce mot m dans la colonne message du tableau et dans la ligne correspondante au syndrome trouvé. Si la place est déjà prise on ne fait rien.
On s'arrête quand la deuxième colonne du tableau est complète.

Le tableau obtenu donne, pour chaque syndrome s, un mot de $\mathbb{B}^n$ de poids minimal dont le syndrome est s.

Retour à l'exemple : Le tableau avec la liste des syndromes est le suivant :

syndrome	message de poids minimal ayant ce syndrome
000	0000000
001	0000001
010	0000010
011	0001000
100	0000100
101	1000000
110	0010000
111	0100000

Méthode pratique de correction :
On reçoit le message m.
On en calcule le syndrome s.
On corrige en ajoutant au message m le message figurant dans la ligne de s dans la liste des syndromes

Retour à l'exemple :
On reçoit le message $m = 0110011$; le calcul du syndrome donne $s = 010$; on corrige en ajoutant à m le vecteur d'erreur 0000010.
Cela donne 0110001

Le tableau suivant donne, pour un certain nombre de messages, les corrections obtenues.

message reçu	syndrome	correction obtenue
0110011	010	0110001
1011101	000	1011101
0110101	010	0110111
...	...	...

6.4 Code de Hamming

Il nous reste à savoir comment construire un codage linéaire efficace. On souhaite qu'il puisse détecter et corriger parfaitement les messages comportant 1 erreur. La distance du code doit donc être au moins égale à $d = 3$.

Théorème : Un code linéaire corrige de façon certaine une erreur si et seulement si la matrice H^t a toutes ses lignes distinctes et non nulles.

Définition : Un **code de Hamming** est un code linéaire $[n, k]$ $(r = n - k)$ avec :
- $n = 2^r - 1$, et $k = 2^r - 1 - r$
- la matrice H^t constituée de tous les mots binaires de longueur r

Propriété : **Un code de Hamming a pour distance $d = 3$ et c'est un code parfait.**

Retour à l'exemple : l'exemple proposé est un code de Hamming.

6.5 Synthèse sur les erreurs ...

$$\underset{M \in \mathbb{B}^k}{\text{Message initial } M} \xrightarrow{\text{codage}} \underset{E \in \mathbb{B}^n}{\text{Message envoyé}} \Downarrow \text{transmission} \underset{R \in \mathbb{B}^n}{\text{Message reçu}} \xrightarrow{\text{décodage}} \underset{M' \in \mathbb{B}^k}{\text{Message final } M'}$$

.

Message envoyé E

↙ ↘

bien transmis
$E = R$

mal transmis
au moins une erreur s'est produite
le message est faux
$E \neq R$

↙ ↘

détecté
R n'est pas un mot de code
$\Rightarrow$ on fait une correction
R donne R'

non détecté
R est un mot de code

↙ ↘

bien corrigé
$R' = E \Rightarrow M' = M$

mal corrigé
$R' \neq E$
et dans ce cas, on peut avoir $M' \neq M$
(mais aussi éventuellement $M' = M$)

7 Eléments de bibliographie

Gilles DUBERTRET. *L'univers secret de la cryptographie.* Vuibert, 2015
→ un contenu court et accessible avec peu de formalisme
→ une bonne introduction mais pas d'exercice

Philippe GUILLOT. *La cryptologie, l'art des codes secrets.* EdpSciences, 2013
→ contenu bien illustré
→ assez complet mais pas d'exercice

Jacques VELU. *Méthodes mathématiques pour l'informatique, cours et exercices corrigés.* Dunod, 2013
→ un livre incontournable pour comprendre les mathématiques liées à l'informatique
→ une bonne présentation des codes correcteurs et des codes détecteurs

Damien VERGNAUD. *Exercices et problèmes de cryptographie.* Dunod, 2021
→ des présentations courtes des principales méthodes
→ de très nombreux exercices et problèmes
→ un contenu complet et adapté à des étudiants qui poursuivraient leurs études dans ce domaine

Pierre VIGOUREUX. *Cryptographie, libertés individuelles et codes secrets.* Ellipses, 2006
→ assez complet mais pas d'exercice

Jean-Philippe AUMASSON. *La cryptographie déchiffrée / Une introduction au chiffrement moderne.* Dunod, 2024
→ intéressant, assez complet, adapté à des étudiants qui souhaiteraient aller plus loin dans ce domaine

Rémi BOULLE. *Cryptographie par la pratique avec Python et OpenSSL.* Dunod, 2024